百原 GEO Platform 技術白皮書

在生成式 AI 時代重新定義品牌可見性的工程實踐

Vincent Lin（百原科技）

2026-04-18

摘 要

本書記錄百原科技於 2024–2026 年開發「百原 GEO Platform」的工程實踐。百原 GEO Platform 是一套針對生成式引擎優化（Generative Engine Optimization, GEO）的 SaaS 系統，目標是協助品牌在 ChatGPT、Claude、Gemini、Perplexity 等生成式 AI 的回答中被正確、持續、準確地提及。本書涵蓋一套七維度 AI 引用率評分演算法、一個 對 AI Bot 友善的影子文檔（AXP）交付機制、Schema.org 三層實體知識圖、以及一個 幻覺自動偵測與修復的閉環系統。採 CC BY-NC 4.0 授權公開。

目錄

Chapter 1 —GEO 生成式引擎優化的時代背景與挑戰 **4**
目錄 5
1.1 生成式搜尋接管使用者習慣 5
1.2 AI 引用率：一個嶄新且決定生死的指標 5
1.3 GEO 不是 SEO 的延伸，是獨立新學科 6
1.4 市場現況：海外先行，台灣空白 7
1.5 本書動機與閱讀路徑 7
本章要點 7
參考資料 7

Chapter 2 —百原 GEO Platform 系統總覽 **8**
目錄 8
2.1 設計哲學：從監測工具到閉環系統 8
2.2 三大模組 8
2.3 核心資料流 9
2.4 技術棧 10
2.5 多租戶資料隔離 10
2.6 AI 平台覆蓋 11
本章要點 11
參考資料 11

Chapter 3 —七維度 GEO 評分演算法 **12**
目錄 12
3.1 為何單一分數不夠 12
3.2 七維度設計 12
3.3 權重設計哲學 14
3.4 函數骨架 14
3.5 與傳統 SEO 分數的對比 15
3.6 演算法的限制 15
本章要點 15
參考資料 16

Chapter 4 —Stale Carry-Forward：訊號連續性與資料新鮮度的工程設計 **17**
目錄 17
4.1 問題定義 17
4.2 為何直覺解法都是錯的 17
4.3 Stale Carry-Forward 設計 18
4.4 設計取捨 19
4.5 資料新鮮度的哲學 19
4.6 函數骨架 19
4.7 延伸適用領域 20
本章要點 21
參考資料 21

Chapter 5 —多 Provider AI 路由：前瞻性容錯架構設計 **22**
目錄 22
5.1 單一供應商的結構性風險 22
5.2 modelRouter 架構 22
5.3 核心工程難題 23
5.4 重試策略疊加陷阱 24
5.5 切換觸發與觀察性 25
5.6 備援路徑的健康管理 26
5.7 函數骨架與資料結構 26
本章要點 28
參考資料 28

Chapter 6 —AXP 影子文檔：用 Cloudflare Worker 對 AI Bot 服務乾淨內容 **29**
目錄 29
6.1 問題：為何同一份 HTML 服務兩方都吃虧 29
6.2 AXP 設計：影子文檔的結構 30
6.3 Cloudflare Worker 注入機制 31
6.4 AI Bot UA 清單與識別策略 32
6.5 SaaS 自家品牌的路徑衝突 33
6.6 Sitemap 自動產生 34
6.7 JSON-LD 扁平化的踩坑紀錄 34
6.8 GSC 索引血淚清單 35
6.9 RAG 知識庫串接：品牌專屬知識庫自動化 35
本章要點 37
參考資料 37

Chapter 7 —Schema.org Phase 1：25 產業 × 三層 @id 互連 **38**
目錄 38
7.1 Schema.org 在 AI 時代的角色重定位 38
7.2 25 類產業特化 @type 的設計 38
7.3 三層 @id 互連知識圖 39
7.4 實體商家 vs 線上服務：欄位權重的分歧 41
7.5 資料完整度演算法 41
7.6 Wizard + Edit 雙入口設計 42
7.7 GBP URL Parser 43
7.8 函數骨架 44
本章要點 45
參考資料 45

Chapter 8 —GBP API 整合策略：從事實主控源到對外輸出的單向同步 **46**
目錄 46
8.1 為何 GBP 是實體商家的事實主控源 46
8.2 GBP API 申請門檻與流程 46

8.3 代管模式選擇 . . . 47
8.4 欄位對應表 . . . 47
8.5 同步頻率與配額 . . . 48
8.6 無 Webhook 的補救方案 . . . 49
8.7 Phase 1–4 Roadmap . . . 49
本章要點 . . . 50
參考資料 . . . 50

Chapter 9 —Closed-Loop 幻覺偵測與自動修復 **51**
目錄 . . . 51
9.1 為什麼「偵測」不足、需要「閉環」 . . . 51
9.2 AI 幻覺的五種類型 . . . 51
9.3 偵測主機制：NLI 分類 + Chainpoll 投票 . . . 52
9.4 中央共用 RAG：SaaS 架構的關鍵基礎設施 . . . 54
9.5 L1 LLM Wiki：被動檢索之上的主動語意層 . . . 55
9.6 修復：ClaimReview 生成與多路徑注入 . . . 56
9.7 兩層掃描閉環 . . . 57
9.8 收斂時序與驗收 . . . 58
本章要點 . . . 59
參考資料 . . . 59

Chapter 10 —Phase 基線測試：以固定問題集做縱向 AI 認知比較 **60**
目錄 . . . 60
10.1 常規掃描的縱向追蹤盲點 . . . 60
10.2 Phase 基線測試設計 . . . 60
10.3 與常規掃描的資料路徑區隔 . . . 61
10.4 四類觀察軸 . . . 62
10.5 實務操作指引 . . . 62
本章要點 . . . 63
參考資料 . . . 63

Chapter 11 —5 品牌實戰觀察：6 週運營的匿名化數據 **64**
目錄 . . . 64
11.1 品牌畫像（匿名） . . . 64
11.2 GEO 分數分布 . . . 64
11.3 平台覆蓋差異 . . . 65
11.4 Schema.org 完整度與引用率的相關性 . . . 66
11.5 AXP 部署前後對比 . . . 66
11.6 客戶端常見踩坑 . . . 67
11.7 三個意外發現 . . . 67
11.8 上線首月的商業驗證 . . . 68
本章要點 . . . 68
參考資料 . . . 68

Chapter 12 —限制、未解問題與未來工作 **70**
目錄 . . . 70
12.1 目前做不到的事 . . . 70
12.2 AI 模型版本變動的不可預測性 . . . 71
12.3 未解問題 . . . 71
12.4 未來工作 Roadmap . . . 72
12.5 給同業與研究者的邀請 . . . 73
本章要點 . . . 73
參考資料 . . . 73

Appendix A —詞彙表（完整版） **74**

A . . . 74
B . . . 74
C . . . 74
D . . . 74
E . . . 74
F . . . 75
G . . . 75
H . . . 75
I . . . 75
J . . . 75
K . . . 75
L . . . 75
M . . . 75
N . . . 75
O . . . 76
P . . . 76
Q . . . 76
R . . . 76
S . . . 76
T . . . 76
U . . . 76
W . . . 76
相關規範索引 . . . 77

Appendix B —公開 API 規格節錄 **78**
共通規範 . . . 78
1. Brands . . . 78
2. Scans . . . 79
3. Entity（Schema.org 品牌實體） . . . 80
4. AXP（影子文檔） . . . 81
5. Hallucinations . . . 81
6. Baseline（Phase 基線測試） . . . 81
7. Webhooks（客戶可接收） . . . 82
錯誤代碼 . . . 82

Appendix C —參考文獻 **83**
1. 官方規範與標準 . . . 83
2. 技術框架與開發工具 . . . 84
3. 產業研究與統計資料 . . . 84
4. 書籍與專書 . . . 84
5. 相關學術研究（選錄） . . . 84
如何引用本書 . . . 85

Appendix D —配圖總表與製圖規格 **86**
D.1 總表 . . . 86
D.2 製圖準則 . . . 87
D.3 製作進度 . . . 89

Chapter 1 —GEO 生成式引擎優化的時代背景與挑戰

使用者不再「搜尋」答案，他們直接「詢問」答案。當檢索被生成取代，品牌的可見性規則就必須重寫。

目錄

- 1.1 生成式搜尋接管使用者習慣
- 1.2 AI 引用率：一個嶄新且決定生死的指標
- 1.3 GEO 不是 SEO 的延伸，是獨立新學科
- 1.4 市場現況：海外先行，台灣空白
- 1.5 本書動機與閱讀路徑
- 本章要點
- 參考資料

1.1 生成式搜尋接管使用者習慣

2023 年以前，「使用者想知道某件事」幾乎等同於「使用者會打開 Google」。2024 年之後，這條假設開始動搖；2025 年底，它正式成為歷史。

根據 OpenAI 2025 年第三季揭露資料[1]，ChatGPT 每日處理的使用者查詢量已超過 **25 億次**；Perplexity 公開的月查詢量為 **7.8 億次**，且年增長維持三位數[2]。Google 自家的 AI Overview 自 2024 年 5 月全量開啟後，使用者對 SERP 傳統藍色連結的點擊率，依 SimilarWeb 與 Digital Research Index 追蹤數據，平均下降 **34–48%**[3]。在高資訊密度領域（醫療、法律、SaaS 選型），下降幅度更接近 **60%**。

對品牌而言，結論只有一個：

> 「使用者找到你」的入口，正從『10 條藍色連結』遷徙到『一段 AI 生成的答案』。如果你的品牌沒有在那段答案裡出現，你基本上等同不存在於那位使用者的決策路徑。

這不是預測，是已經發生的事。

Fig 1-1：搜尋管道流量遷徙（示意）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Information Retrieval Channel Share (2023–2025, illustrative)"
    x-axis ["2023 Q1", "2023 Q4", "2024 Q2", "2024 Q4", "2025 Q2", "2025 Q4"]
    y-axis "Share of Queries (%)" 0 --> 100
    bar [85, 78, 65, 52, 42, 35]
    line [5, 10, 18, 28, 36, 42]
```

Fig 1-1: 藍色長條為「傳統 Google SERP」，折線為「生成式 AI（含 AI Overview）」合計占比。實際數字依研究機構與地區有差異，此圖為趨勢示意。

1.2 AI 引用率：一個嶄新且決定生死的指標

當使用者問 ChatGPT、Claude、Perplexity、Copilot、Gemini：

- 「有哪些推薦的 B2B 行銷自動化工具?」
- 「台北信義區評價最好的醫美診所是哪幾間?」
- 「CRM 類型 SaaS 裡哪個比較適合中小企業?」

AI 會**生成一段包含具體品牌名稱的回答**。被提到的品牌自然進入使用者的候選清單；沒有被提到的品牌**根本不會出現在對話裡**。使用者不會再追問「還有別家嗎?」，就如同他們過去不會翻到 Google 搜尋結果的第 5 頁。

[1] OpenAI. (2025). Usage & Revenue Update, Q3 2025. 官方季度揭露。
[2] Perplexity AI. (2025). Year in Review 2024: Search Volume & Engagement. 官方部落格。
[3] SimilarWeb. (2025). The State of Generative Search: AI Overview Impact on Publisher Traffic. 研究報告。

我們把這個現象量化為一個指標：**AI 引用率**（AI Citation Rate）—在一組代表性意圖查詢中，品牌被 AI 主動提及的比例。它不是點擊率、不是展示次數、也不是排名，而是：**你有沒有被 AI「記住」並「提起」。**

這個指標的特性與傳統 SEO 指標截然不同：

特性	傳統 SEO 指標	AI 引用率
透明度	有可被拆解的規則（PageRank、Core Web Vitals）	黑盒（無公開規則）
跨平台	Google 主宰，多數指標可移植	分歧大，ChatGPT/Claude/DeepSeek 各自一套
時間穩定性	演算法更新約每季	模型重訓可能每週變動
輸出形式	可點擊連結	自然語言敘述（可好可壞）

換言之，被 AI「連結」不重要，被 AI「描述」才重要。描述得對不對、好不好，就是品牌資產本身。

1.3 GEO 不是 SEO 的延伸，是獨立新學科

若用一句話劃定界線：**SEO 是讓 Google 把你排在第 1 名；GEO 是讓 AI 在生成回答時主動提到你。**

Fig 1-2：SEO vs GEO 核心差異對照

維度	SEO	GEO
成功結果形式	一條可點擊的藍色連結	一段自然語言敘述中的品牌名稱
觸發條件	關鍵字匹配 + 權威信號 + UX 訊號	模型訓練與檢索增強中的「實體關聯強度」
可操控槓桿	內容關鍵字、外鏈、結構化資料、Core Web Vitals	結構化實體資料、可信來源、幻覺修復、AI Bot 爬蟲相容性
驗收指標	排名、點擊率、停留時間	引用率、位置品質、敘事情感、跨平台一致性
時間尺度	幾週到幾月	模型重訓週期（通常以季為單位）
主要受眾	人類瀏覽器	AI 模型 + 其爬蟲/檢索管道

Fig 1-2: SEO 與 GEO 並列但規則不同；任何把 GEO 當 SEO 子議題的規劃都會導致資源錯配。

很多傳統 SEO 從業者會說：「做好 SEO，AI 自然會引用你」。這句話在 2023 年部分成立，在 2025 年完全失效。原因有二：

其一，AI 的資料來源已不再是 Google 索引的投射。主流大型語言模型在預訓練階段消化 Common Crawl、專業出版物、開源知識庫（Wikipedia、Wikidata），以及各家廠商自建的檢索增強資料集。Google 排名在此流程中僅占一小部分訊號，且為間接訊號。

其二，結構化資料對 AI 的權重遠高於對傳統搜尋的權重。AI 理解實體（entity）靠的不是 H1/H2 與關鍵字密度，而是 Schema.org JSON-LD、Wikidata triples、以及可對應的知識圖譜節點。一個 SEO 分數滿分但沒有 Schema.org 結構的網站，在 AI 眼中幾乎等同空白。

GEO 不是 SEO 的下一個版本，它是一個**與 SEO 並列但規則不同的學科。**不接受這個前提，就無從規劃相關的工程投資。

1.4 市場現況：海外先行，台灣空白

海外在 2024 年已經出現第一批 GEO 工具商：

- **Profound**（美國）—專注 Fortune 500，主打跨 AI 平台的品牌監測與內容優化建議
- **Otterly.ai**（歐洲）—中小企業取向，提供 AI 引用率儀表板與競品對比
- **AthenaHQ**（以色列）—強調企業級 RAG 接入與 AI 內容審核

這些工具主打英文市場，介面、AI 模型覆蓋、知識圖譜資料都以歐美為中心。對中文市場的支援（繁體中文語料、台灣本地品牌知識、中國模型如文心/DeepSeek/Kimi/智譜的覆蓋）基本缺席。

台灣在同期的自有解決方案近乎為零。多數品牌主、行銷代理商甚至仍在用「Google 搜尋你品牌名看第幾個結果」這種手工驗證方式；偶爾在 ChatGPT 裡試問一句「有哪些 X 類品牌」，然後人工記下結果。這是一個正在擴大的市場空白。

1.5 本書動機與閱讀路徑

百原 GEO Platform 是為了填補這個空白而誕生的工程專案。從 2024 年第一版原型到 2026 年目前服務的產品版本，我們累積了一組值得被白皮書化的工程經驗——涵蓋演算法、架構、容錯設計、對 AI Bot 友善的內容交付、結構化實體管理、幻覺自動修復，以及一套在多租戶 SaaS 下可持續運行的資料治理模式。

本書不是產品宣傳，也不是使用說明，而是一份**工程實踐報告**：揭露我們為何如此設計、哪些選擇踩過坑、哪些模式可被其他團隊複用。

本書服務三類讀者：

1. **B2B 決策者** —建立「AI 時代品牌可見性」的認知框架，避免把 GEO 錯當 SEO 子議題
2. **工程主管與架構師** —借鑑「多 AI Provider 容錯」「訊號連續性」「閉環自動修復」等模式
3. **開發者與技術人** —看到 Schema.org、Cloudflare Worker、pgvector、BullMQ 等工具在真實產品中如何被組裝

接下來 11 章將依序覆蓋系統總覽、核心演算法、對外可見性建構、質量保證閉環，以及實戰數據與反思。凡涉及客戶個資與商業敏感數字之處，一律以聚合或去識別化呈現；凡涉及演算法細節之處，骨架完整揭露、具體權重數字保留——這是在知識分享與商業利益之間我們能找到的平衡點。

本章要點

- 生成式 AI 搜尋已實質改變使用者資訊取得管道，傳統 SERP 點擊率下降 34–60%
- 「AI 引用率」是 GEO 的核心指標，與 SEO 排名性質不同、機制不同、可操控槓桿不同
- GEO 與 SEO 並列而非相繼；把 GEO 當 SEO 子議題會導致資源錯配
- 海外已有 Profound、Otterly、AthenaHQ 等先行工具，中文市場尚無完整解決方案
- 本書為百原 GEO Platform 的工程實踐報告，服務決策者、工程主管、開發者三類讀者

參考資料

Chapter 2 —百原 GEO Platform 系統總覽

百原 GEO Platform（下稱「百原 GEO」或「本平台」）是本書主角，由百原科技（Baiyuan Technology）開發與維運。本章是後續 10 章的地圖索引。

目錄

- 2.1 設計哲學：從監測工具到閉環系統
- 2.2 三大模組
- 2.3 核心資料流
- 2.4 技術棧
- 2.5 多租戶資料隔離
- 2.6 AI 平台覆蓋
- 本章要點
- 參考資料

2.1 設計哲學：從監測工具到閉環系統

市場上第一代 GEO 工具多數停留在**監測**層面：提供一個儀表板，告訴使用者「你現在幾分」。問題是，監測只是診斷，不是治療。使用者看到低分之後會問：「然後呢?」

百原 GEO 的設計起點，是把整個流程視為一個**閉環**（Closed Loop）：

Fig 2-1：閉環系統的六階段

```
flowchart LR
    A[監測 <br/>Scan] --> B[診斷 <br/>Diagnose]
    B --> C[優化 <br/>Optimize]
    C --> D[部署 <br/>Deploy]
    D --> E[驗證 <br/>Verify]
    E --> A
    E -.->| 幻覺偵測 | F[修復 <br/>Remediate]
    F --> D
```

Fig 2-1: 每一環節都必須可被自動化、可被量化、可被追溯。任何必須「人工做一次」的環節都是系統的疤痕。

基於這個原則，我們把系統切分為三個模組。

2.2 三大模組

2.2.1 Scan Module —監測

負責**週期性地把品牌送去問 AI**，把回應抓回來、結構化、評分。

核心職責

- 意圖查詢生成：依品牌產業動態產生 20–60 則代表性問題
- 多平台派發：同一組問題分發到 15+ 個 AI 平台與搜尋引擎
- 回應抽取：從自然語言回答中抽出「品牌是否被提及」「提及位置」「情感語氣」「競品共現」
- 評分計算：依七維度演算法合成 GEO 總分（見 Ch 3）
- 訊號維持：遇到平台中斷時啟用 Stale Carry-Forward 機制（見 Ch 4）

技術選型

- 任務隊列：**BullMQ** on Redis，支援重試、優先級、速率限制
- AI 路由：自研 **modelRouter** 服務，主路徑 OpenAI-compatible 中轉 + 各廠商直連 fallback（見 Ch 5）
- 掃描模式：常規週期（daily）+ 哨兵（4h，見 Ch 9）+ Phase 基線（週 / 雙週，見 Ch 10）

2.2.2 Visibility Module —對外可見性

負責**讓 AI 更容易認識、引用這個品牌。**

核心職責

- **結構化實體資料管理**：Schema.org JSON-LD，25 產業特化 × 三層 `@id` 互連（見 Ch 7）
- **AXP 影子文檔**：為 AI Bot 產生 pure HTML + JSON-LD + Markdown 的「乾淨版內容」，與人類使用者看的網站解耦（見 Ch 6）
- **CF Worker 注入**：於網路邊緣偵測 AI Bot UA，動態回傳影子文檔或透傳原站
- **GBP 資料整合**：作為實體商家的「事實主控源」，單向同步到 Schema.org（見 Ch 8）
- **知識源建構**：串接 Wikipedia / Wikidata / LinkedIn 等可被 AI 抓取的權威平台

2.2.3 Quality Module —質量保證

負責**偵測並修復 AI 對品牌的錯誤認知**，讓閉環收斂。

核心職責

- **幻覺偵測**：從 AI 回應中抽取「關於品牌的聲明」與 Ground Truth 比對
- **ClaimReview 生成**：為每個已確認的幻覺產生 Schema.org ClaimReview 結構
- **知識庫同步**：把修正資訊推送到 RAG 系統，讓後續檢索能覆蓋該錯誤
- **兩層掃描閉環**
  - Layer 1 哨兵（4h / 搜尋型平台）—快速驗證修復是否被抓取
  - Layer 2 完整（24h / 知識型平台）—含分數與指紋驗證的深度確認

2.3 核心資料流

Fig 2-2：一次完整的品牌治理循環

```
flowchart TD
    U[使用者建立品牌] --> W[Wizard 引導 Schema.org 填寫]
    W --> G[系統產出 AXP 影子文檔 + JSON-LD]
    G --> CF[CF Worker 對 AI Bot 提供影子內容]
    CF --> S[Scan Module 派發意圖查詢 <br/> 到 15+ AI 平台]
    S --> R[回應回流 → 評分]
    R --> D[儀表板呈現七維度 <br/>+ 競品對比 + 缺口分析]
    R --> Q[Quality Module 偵測幻覺]
    Q --> C[生成 ClaimReview<br/> 注入 AXP + RAG]
    C --> V[哨兵 / 完整掃描驗證]
    V -->| 收斂 | S
    V -.->| 仍有問題 | C
```

Fig 2-2: 閉環可以以「小時」為單位（哨兵）或以「日」為單位（完整掃描）運作；使用者看到的只是儀表板上的分數變化，背後是上述八個步驟的持續循環。

2.4 技術棧

層級	技術	主要職責
Edge	Cloudflare Workers	AI Bot UA 偵測、影子文檔注入、sitemap / robots 代管
Frontend	Next.js 16（Webpack）+ React 19 + TypeScript + Tailwind v4	儀表板、品牌管理、Wizard、i18n（zh-TW / en）
API	Node.js + Express 4 + Helmet + express-rate-limit + Zod	REST API、多租戶、JWT + 2FA
Worker	BullMQ 5 + Node.js 任務執行程序	掃描、評分、AXP 生成、幻覺偵測、RAG 同步
AI Routing	自研 modelRouter + OpenAI SDK + 各廠商直連 SDK	多 Provider 容錯
Data	PostgreSQL 16 + pgvector + Redis 7	關聯資料、向量檢索、快取、隊列
Deploy	Docker Compose on AWS Lightsail（PROD）/ 本機 Docker（UAT）	環境隔離
RAG	中央共用 RAG 引擎（內部 SaaS 基礎設施）	多租戶知識庫同步與查詢

截至 2026-04-18，系統已演進至 v2.19.4；累計功能迭代約 20 個 minor version、migration 檔案 139 支、前端頁面 60+、API endpoint 180+。

2.5 多租戶資料隔離

百原 GEO 是 B2B SaaS，同一個資料庫同時服務多組客戶，**資料隔離是合約等級的義務**，不是一個「最好有」的 nice-to-have。我們採用**雙保險**策略：

1. **Row-Level Security（RLS）on PostgreSQL**：在 `brands`、`scan_results`、`geo_scores` 等核心表建立 RLS 政策，依 `current_setting('app.org_id')` 過濾；即使應用層寫錯 SQL，資料也不會跨組洩漏
2. **App-level filter**：每個 query 層的 API 仍然顯式帶上 `org_id`，與 RLS 形成雙層驗證；若 RLS 政策被誤刪，應用層仍能保障隔離

Fig 2-3：雙保險示意

```
flowchart LR
    R[HTTP Request] --> M[Auth Middleware<br/> 解析 org_id]
    M --> C[Controller]
    C --> Q[Query Layer<br/> 顯式帶 WHERE org_id]
    Q --> PG[(PostgreSQL)]
    M -.->|SET app.org_id| PG
    PG --> RLS{RLS Policy<br/> 依 current_setting 過濾}
    RLS --> D[(Data)]
```

Fig 2-3: 應用層與資料庫層各自獨立執行一次過濾。單一層出錯不會造成跨組洩漏。

此設計來自早期一次接近失誤的事件：僅靠應用層帶條件，人類會忘記；僅靠 RLS，政策變更時無法被一般 CI 偵測。雙保險讓兩種疏漏都需要同時發生才會出問題。

2.6 AI 平台覆蓋

截至本書撰寫時，百原 GEO 覆蓋 **15 個 AI 平台**，分成三群：

類型	數量	代表平台
全球通用型	7	OpenAI GPT-4o / 4o-mini、Anthropic Claude、Google Gemini、Meta Llama、Mistral、xAI Grok、Cohere
中文模型	5	百度文心、DeepSeek、月之暗面 Kimi、智譜 ChatGLM、阿里通義千問
搜尋型	3	Perplexity、ChatGPT Search、Google AI Overview（爬蟲側）

覆蓋一個平台的成本不僅是「串一個 API」，還包括：model ID 映射、extraParams 設計、timeout / retry 策略、幻覺偵測模板、評分校準、UI 呈現。每加一個平台約需 2–4 個工程日。

本章要點

- 百原 GEO 以「閉環」設計為起點，監測、診斷、優化、部署、驗證五環皆自動化
- 系統切分為 Scan / Visibility / Quality 三大模組，各自對應後續章節
- 技術棧以 Next.js 16 + Express + BullMQ + PostgreSQL 16 + pgvector + Redis 為核心
- 多租戶隔離採 RLS + App-level filter 雙保險，容錯單一層人為失誤
- 覆蓋 15 個 AI 平台（全球 7 + 中文 5 + 搜尋型 3），每加一平台成本 2–4 工程日

參考資料

- Ch 3 —七維度評分演算法
- Ch 4 —Stale Carry-Forward
- Ch 5 —多 Provider AI 路由
- Ch 6 —AXP 影子文檔
- Ch 7 —Schema.org Phase 1
- Ch 8 —GBP API 整合
- Ch 9 —Closed-Loop 幻覺修復
- Ch 10 —Phase 基線測試

Chapter 3 —七維度 GEO 評分演算法

> 一個單一分數可以把任何複雜系統壓扁成一個可比較的數字；但也可能把該被看見的差異一起壓扁。

目錄

- 3.1 為何單一分數不夠
- 3.2 七維度設計
- 3.3 權重設計哲學
- 3.4 函數骨架
- 3.5 與傳統 SEO 分數的對比
- 3.6 演算法的限制
- 本章要點
- 參考資料

3.1 為何單一分數不夠

百原 GEO 在 2024 年的第一版評分，只有一個指標：**Citation Rate**，計算方法是「被提及次數 ÷ 總查詢次數」。簡單、直觀、可對比。然而上線三個月後，我們發現這個單一指標會產生大量**誤導相同**的案例。

以下兩個真實案例在 v1 下都得到 55 分，但品牌體質完全不同：

- **品牌 A**：在 20 個查詢中被提及 11 次，總是被放在回答的末段、以「此外也有 □□」的形式點名；所有提及分布在 OpenAI 與 Anthropic 兩家
- **品牌 B**：在 20 個查詢中被提及 11 次，有 8 次出現在回答的第一、二句；分布在 7 個不同 AI 平台，且附帶具體描述

這兩個品牌的實際「被 AI 認知的能量」差距極大，但單一分數把它們壓成同一個格子。使用者若只看這個分數，會錯失所有可行動的資訊。

單一分數的根本問題在於：**GEO 是一個多維度的現象**。被提到的次數、提到的位置、提到的平台、提到的語氣、描述的深度、跨平台的一致性——這些都是獨立的訊號，混成一個平均值之後資訊全部遺失。

3.2 七維度設計

百原 GEO v2 的評分把「品牌在 AI 生態的狀態」拆成七個獨立維度：

Fig 3-1：七維度雷達比較（v1 vs v2 示意）

```
%%{init: {'theme':'base'}}%%
graph LR
    A[Citation Rate<br/> 被提及率] --- B[Position Quality<br/> 位置品質]
    B --- C[Query Coverage<br/> 查詢覆蓋率]
    C --- D[Platform Breadth<br/> 平台覆蓋度]
    D --- E[Sentiment<br/> 情感分數]
    E --- F[Content Depth<br/> 內容深度]
    F --- G[Consistency<br/> 跨平台一致性]
    G --- A
```

Fig 3-1: 七維度關聯示意；相鄰維度有資訊互補性，非相鄰維度獨立計算。實際雷達圖（各維度 0–100）留待 PDF 版以真實數據呈現。

3.2.1 Citation Rate —被提及率

定義：在代表性意圖查詢中被品牌名主動提及的比例（0–100）。

計算起點：`mentioned_count / query_count`。

精煉：

- 去除假陽性：競品同名、相似產品線誤認
- 加入 lemma 比對：品牌全名、簡稱、英文名、中英混寫都算同一個提及
- 跨平台先算各平台率再加權平均，避免「某平台次數多」主宰總分

權重佔比：最大單一維度，但刻意壓在 25%（四分之一），避免退回 v1 單一指標。

3.2.2 Position Quality —位置品質

定義：品牌在 AI 回答中出現位置的加權平均分數。

計算邏輯：把 AI 的回答切成段、句、列點；每個「提及」依出現位置給加權：

位置	權重
首句 / 列表第 1 項	1.0
前三分之一	0.8
中段	0.5
末段 / 延伸補充	0.2

此維度解答了 v1 時代的常見疑問：「為什麼我被提到但感覺沒幫助?」答案往往是：提得太晚、太輕描淡寫。

3.2.3 Query Coverage —查詢覆蓋率

定義：被提及的查詢類型多樣性。

計算邏輯：將意圖查詢分類（「最佳選擇型」「比較型」「問題解法型」「入門推薦型」），統計被提及的覆蓋類型數 ÷ 總類型數。

此維度揭露：有些品牌只在「最佳 X 工具」類問題中被提及，但在「我該選 A 還是 B」的比較題中完全消失——這是不同的戰場，需要不同的內容策略。

3.2.4 Platform Breadth —平台覆蓋度

定義：品牌被多少個 AI 平台主動提及。

計算邏輯：出現提及的平台數 ÷ 監測平台總數。

此維度揭露平台偏食。例如某 SaaS 只在 OpenAI 系列被提及，但在中國模型（DeepSeek、Kimi）完全缺席；這不是隨機誤差，而是指向特定的內容可見性問題（訓練資料來源、結構化資料覆蓋度）。

3.2.5 Sentiment Score —情感分數

定義：提及文本的情感傾向。

計算邏輯：用獨立的情感分類模型把每次提及分為「正面 / 中性 / 負面」，聚合成 0–100 分。不直接用 AI 的原 reasoning，避免「A AI 對 A AI 的自我評價」問題。

此維度在品牌有危機事件後特別敏感。一般掃描下中性佔大多數（70–80%），當跌到 50% 時通常意味著外部出現了可檢視的負面內容源。

3.2.6 Content Depth —內容深度

定義：AI 提及品牌時所附帶的描述深度。

計算邏輯：分析該段「關於品牌的自然語言文字」的長度、實體密度（提到多少個相關事實—產品線、創辦人、應用情境等）、句法複雜度。

此維度區分了「被點名」（一句話帶過）與「被介紹」（一段深度描述）。對於 B2B SaaS、教育機構、專業服務，Content Depth 比 Citation Rate 本身更關鍵——深度描述才能真正帶來轉換。

3.2.7 Consistency —跨平台一致性

定義：同一品牌在不同 AI 平台的上述六維分數的標準差之倒數（正規化到 0–100）。

計算邏輯：一致性高的品牌，在 ChatGPT、Claude、Gemini、DeepSeek 都得到相近的敘事與分數；一致性低的品牌，可能在某平台形象鮮明、在另一平台模糊不清。

此維度本身不改變其他維度，但它提供「結果可靠性」的訊號：一致性高的分數代表品牌的 AI 認知已經收斂到一個穩定的實體印象；一致性低則代表 AI 間對此品牌的認知仍在分歧。

3.3 權重設計哲學

七個維度不是平均加總。各維度的權重以三個準則取捨：

1. **重要性** —Citation Rate 與 Content Depth 是最直接連動「品牌被認識」的兩個指標，權重較高
2. **雜訊敏感度** —Sentiment 與 Consistency 容易受單次回應異常影響，權重較低以避免鋸齒
3. **可操控度** —Platform Breadth 與 Query Coverage 是可被「主動行動」直接改善的，給一定權重以讓分數能對改善產生回應

本章**揭露公式骨架與權重分群（高 / 中 / 低）**，不揭露精確數字。這個決定不是為了保密，而是為了**防止客戶優化指標而非優化實質**：

> 如果你知道某一維度佔 30%，你就會砸資源只衝那一維度；結果分數上升、AI 實際認知卻沒變。我們寧願讓你專注於「改善整體內容質量」而非「拆解分數權重」。

此設計哲學與搜尋引擎演算法的歷史一致——Google 從未公開 PageRank 的精準權重，原因相同。

3.4 函數骨架

```
// weights are resolved from config; values are deliberately not exposed.
function calcGEOScore(scanResults, brandId) {
  const dims = {
    citation:    computeCitationRate(scanResults, brandId),
    position:    computePositionQuality(scanResults, brandId),
    coverage:    computeQueryCoverage(scanResults, brandId),
    breadth:     computePlatformBreadth(scanResults, brandId),
    sentiment:   computeSentimentScore(scanResults, brandId),
    depth:       computeContentDepth(scanResults, brandId),
    consistency: computeConsistency(scanResults, brandId),
  };

  const weighted =
    dims.citation    * W_CITATION +
    dims.position    * W_POSITION +
    dims.coverage    * W_COVERAGE +
```

```
    dims.breadth     * W_BREADTH +
    dims.sentiment   * W_SENTIMENT +
    dims.depth       * W_DEPTH +
    dims.consistency * W_CONSISTENCY;

  return {
    total: Math.round(weighted),
    dimensions: dims,
    version: SCORING_VERSION,
  };
}
```

3.5 與傳統 SEO 分數的對比

面向	SEO Score	GEO Score
輸入資料	網頁內容 + 外鏈 + UX metrics	AI 回應文本 + 實體比對 + 跨平台聚合
結果型態	連結排名	自然語言提及
時間粒度	每日	每日 + 哨兵 4h + Phase 週
可比性	跨網站可比	跨產業不嚴格可比（不同產業查詢空間不同）
版本敏感度	低	高（受 AI 模型版本影響）

這張表強調一件事：**GEO Score 是「品牌狀態指標」，不是「品質排名指標」。**不同產業的分數基線不同，跨產業比大小並無意義。

3.6 演算法的限制

誠實列出這個七維度系統做不到的事：

- **不反映商業結果**：GEO 分數高不等於轉換好；它是 AI 可見性的代理指標，不是業績指標
- **受模型版本重訓影響**：OpenAI 或 Claude 發布新版可能讓分數整體跳動 3–10 分，這是外部變動不是品牌變動
- **查詢空間主觀**：同產業的兩套查詢集可能產生不同分數；我們透過固定基線（Ch 10）緩解，但無法消除
- **中文模型覆蓋尚在擴充**：中國模型的情感模型、位置偵測準確度仍低於英文模型

這些限制不是演算法的失敗，而是對 GEO 本質的誠實陳述。任何聲稱「能精準量化 AI 認知」的工具都應該被懷疑。

本章要點

- 單一 Citation Rate 會把體質不同但總分相同的品牌誤判為相同
- 七維度（Citation / Position / Coverage / Breadth / Sentiment / Depth / Consistency）各有獨立訊號與用途
- 權重以「重要性 / 雜訊敏感度 / 可操控度」三準則配置；精確數字刻意不揭露以防止指標囚徒困境
- GEO Score 是品牌**狀態**指標、不是**品質**排名，跨產業比大小無意義
- 評分受 AI 模型版本重訓影響，需以 Phase 基線測試作縱向穩定化

參考資料

- Ch 4 —Stale Carry-Forward：訊號連續性設計
- Ch 10 —Phase 基線測試
- Google Search Central. How Search works. https://www.google.com/search/howsearchworks/
- Schema.org. ClaimReview schema. https://schema.org/ClaimReview

---

Chapter 4 —Stale Carry-Forward：訊號連續性與資料新鮮度的工程設計

> GEO 分數反映的是「品牌在 AI 認知中的狀態」，不是「資料管道的健康度」。把兩件事混為一談，分數就失去了它該有的意義。

目錄

- 4.1 問題定義
- 4.2 為何直覺解法都是錯的
- 4.3 Stale Carry-Forward 設計
- 4.4 設計取捨
- 4.5 資料新鮮度的哲學
- 4.6 函數骨架
- 4.7 延伸適用領域
- 本章要點
- 參考資料

4.1 問題定義

任何依賴外部 AI 服務的系統，在任何時點都可能遇到**部分或全部失敗**。失敗的來源五花八門：

- 網路層級的丟封包、CDN 路徑切換、DNS 抖動
- API 層級的 5xx、429 限流、token 配額用盡
- 模型層級的特定版本下線、伺服器重啟、region outage
- 帳戶層級的信用卡過期、額度耗盡、帳單系統延遲

這些失敗在工程上是**日常事件**，而非「異常例外」。問題是：當這些失敗發生在掃描管道時，**不能讓它污染 GEO 分數**。

假想一個場景：品牌 X 的 Citation Rate 在過去 30 天穩定維持在 55 分。某日凌晨，負責為 X 掃描的 6 個 AI 平台中的 3 個同時失敗；若以「失敗即 0」直接計入當日分數，X 會從 55 分掉到 27.5 分。使用者打開儀表板會以為品牌突然失寵、開始恐慌。但實際上 AI 對 X 的認知沒有任何變化——變動的只是資料管道。

這是**第一種錯誤**：把管道故障當成品牌變化。

4.2 為何直覺解法都是錯的

面對上述問題，第一反應往往是以下三種之一，但三種全部有問題：

方案 A：動態分母（剔除失敗平台）

想法：只把成功的平台列入計算，失敗的平台跳過。

問題：分母變動會讓分數**鋸齒狀跳動**。今天 10 個平台全成功、分數 60；明天 6 個成功、分數 62（因為剛好失敗的是低分平台）；後天 10 個全成功、分數又回 60。趨勢圖完全失去可讀性，使用者無法判斷「究竟是真的變好還是分母變了」。

方案 B：失敗即 0%

想法：把每個失敗平台的當日分數記為 0，維持分母穩定。

問題：這等於告訴演算法「這個品牌今天在這個平台完全不存在」，但真實情況是「我們不知道今天的狀態」。**「沒被提及」與「沒掃描到」是兩件完全不同的事**，混為一談會讓後續的趨勢分析、幻覺偵測、競品比較全部失準。

方案 C：靜默丟棄（不寫紀錄）

想法：既然掃描失敗，那就當作這個時段從未存在過。

問題：時序資料庫會出現**斷層**，無法區分「週末沒掃描」「掃描失敗」「掃描了但無變化」三種情境。補救的代價是大量事後判斷邏輯，複雜度高、可維護性低。

這三個解法都有一個共同的根本問題：**它們都沒有區分「資料缺失」與「資料為零」這兩個根本不同的狀態。**

4.3 Stale Carry-Forward 設計

我們採用的方案是**從歷史 carry forward 最近一次的成功值**，並明確標記為「已停滯」：

Fig 4-1：三種策略對同一中斷事件的訊號表現

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Score under Mid-Scan Platform Outage (Illustrative)"
    x-axis ["Day 1", "Day 2", "Day 3 (outage)", "Day 4", "Day 5"]
    y-axis "GEO Score" 0 --> 80
    line [55, 56, 27, 28, 56]
    line [55, 56, 55, 55, 56]
```

Fig 4-1: 上折線 = 方案 B「失敗即 0」造成的假暴跌；下折線 = Stale Carry-Forward 維持連續並於前端標記 isStale。

演算法步驟

1. **偵測**：某平台於當前掃描中失敗率達到 **100%**（所有 query 均無回應或全部 timeout）
2. **查找歷史**：從該品牌該平台的掃描歷史中，**往回查最多 200 列**，找最近一次「非零成功」的 sov_score
3. **Carry forward**：將該歷史值帶入當日紀錄；同時設定兩個標記欄位
   - `isStale = true`
   - `lastSuccessAt = <historical_timestamp>`
4. **前端誠實告知**：儀表板在該平台分數上顯示紅色徽章，hover tooltip 寫「⚠ 已失聯 N 小時，顯示上次成功值」
5. **下游演算法繼續正常運作**：後續的 Consistency、趨勢分析、競品比較都能使用這個 carry-forward 值，不會出現斷層

Fig 4-2：平台資料狀態機

```
stateDiagram-v2
    [*] --> Fresh
    Fresh --> Stale: 當日掃描 100% 失敗
    Stale --> Fresh: 下次掃描成功
    Stale --> Expired: 停滯超過閾值（如 72h）
    Expired --> Fresh: 成功掃描
    Expired --> Reset: 超過 30 天仍未恢復
    Reset --> Fresh: 重新建立基線
```

Fig 4-2: Fresh → Stale 是常態恢復路徑；Expired 與 Reset 是例外處理，留給長期中斷場景。

4.4 設計取捨

4.4.1 為何 lookback 是 200 列而非無限

理由：避免在罕見情況下引用過期資料。假設某平台長期無回應，最早的歷史可能追溯到幾個月前；那時的分數已與當前品牌狀態脫鉤。200 列（對每日掃描而言約 6–7 個月）作為軟上限，在「maintain continuity」與「keep data relevant」之間取平衡。超過此上限則不 carry forward，改記為 null 並在 UI 明確標示「資料待重建」。

4.4.2 首掃品牌沒有歷史怎麼辦

策略：首掃的品牌若遇上平台失敗，**不 carry forward**。因為沒有歷史基線可比；強行帶入 0 或平均值都是假資料。前端改顯示「首次掃描—資料建置中」提示，明確區別「缺歷史」與「有歷史但停滯」。

4.4.3 與 Phase 基線測試的互動

Phase 基線測試（見 Ch 10）走**獨立資料路徑**，不受 Stale Carry-Forward 影響。基線測試的目的是縱向追蹤真實變化，任何 carry forward 都會污染此目的。若基線測試遇上平台失敗，該 Phase 結果直接標示 `status = incomplete`，待恢復後重跑。

4.4.4 Stale 持續多久應升級為警示

Stale 標記本身不是警示；但若**同一平台停滯超過 72 小時**，系統升級通知：

- 前端徽章由紅色常駐變為紅色脈動
- Tooltip 文字改為「⏱ 長時間失聯，請檢查配置」
- 觸發 email 通知給該品牌的管理員

超過 7 天仍未恢復，進一步進入 `Expired` 狀態，於 UI 明示「資料已過期，不納入當前評分」，同時停止 carry forward。

4.5 資料新鮮度的哲學

carry forward 的正當性建立在一個假設上：**AI 認知的變化以週為單位**。模型重訓、知識圖譜更新、外部新聞事件影響，這些都不是小時級的變動。因此用昨天的值替代今天（在資料缺失的情況下）是**統計上合理**的。

但這個假設有邊界：

- **對高頻變動的指標**（如社群聲量、Google Trends），carry forward 超過 24 小時就失準
- **對低頻穩定的指標**（如品牌的 Schema.org 完整度、GBP 驗證狀態），carry forward 可以容忍幾週
- **對當前正在發生的事件**（如 PR 危機、產品發布），carry forward 的掩蓋效應反而是風險

本平台將 carry forward 限制在 **AI 引用率相關的維度**（Citation、Position、Sentiment），不套用於結構化資料狀態、fingerprint 比對等其他指標。

4.6 函數骨架

```
// Simplified illustration — actual implementation handles multi-platform fanout,
// lookback tuning, and emits observability events.
async function enrichWithStaleCarryForward(platform, brandId, currentResult) {
  const currentFailed = currentResult.successCount === 0;
```

```
  if (!currentFailed) {
    return { ...currentResult, isStale: false };
  }

  const lastSuccess = await db.query(`
    SELECT sov_score, position_quality, sentiment, scanned_at
      FROM scan_results
     WHERE brand_id = $1 AND platform = $2
       AND sov_score IS NOT NULL AND sov_score > 0
  ORDER BY scanned_at DESC
     LIMIT 1
    OFFSET 0
  `, [brandId, platform]);

  if (!lastSuccess.rows.length) {
    return { ...currentResult, isStale: false, reason: 'no_baseline' };
  }

  const historical = lastSuccess.rows[0];
  const ageHours = (Date.now() - historical.scanned_at.getTime()) / 3_600_000;

  if (ageHours > MAX_CARRY_FORWARD_HOURS) {
    return { ...currentResult, isStale: true, expired: true };
  }

  return {
    sov_score:        historical.sov_score,
    position_quality: historical.position_quality,
    sentiment:        historical.sentiment,
    isStale:          true,
    lastSuccessAt:    historical.scanned_at,
    staleAgeHours:    ageHours,
  };
}
```

Fig 4-3：前端 isStale 徽章示意

```
OpenAI GPT-4o      55 分    已失聯 14 小時
Anthropic Claude   62 分
Google Gemini      48 分    已失聯 14 小時
```

Fig 4-3: 紅色圓點與 tooltip 明確告知「這個分數是上次成功值，不是當前值」；使用者不會被誤導。

4.7 延伸適用領域

Stale Carry-Forward 這個模式並非 GEO 獨有。任何「高頻取樣、來源不穩、需維持時序連續性」的訊號系統都適用：

領域	適用場景	需調整的參數
IoT 感測	感測器間歇性失聯	lookback window 改以「分鐘」為單位

領域	適用場景	需調整的參數
金融行情	交易所短暫斷線	不適用（金融資料的即時性無法妥協）
社群監測	API 配額用盡	lookback 可更短，變動性高
廣告歸因	Pixel 暫時 loss	需搭配 probabilistic matching
供應鏈可見性	EDI 傳輸中斷	可 carry forward 較長（幾天）

共通前提是：**變動速率遠慢於取樣速率。**一旦違反這個前提（例如金融），carry forward 就不成立。

本章要點

- Stale Carry-Forward 的核心是**分離「品牌狀態」與「管道健康度」**兩種訊號
- 三種直覺解法（動態分母 / 失敗即 0 / 靜默丟棄）都會破壞時序資料的可讀性
- 設計包含偵測、歷史查找、標記、前端誠實告知四步驟；UI 不得隱瞞 stale 狀態
- lookback 上限、首掃處理、與 Phase 基線的分工是三個關鍵取捨
- 此模式可推廣到任何「變動速率 « 取樣速率」的訊號系統

參考資料

- Ch 3 —七維度 GEO 評分演算法
- Ch 5 —多 Provider AI 路由
- Ch 10 —Phase 基線測試
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O’Reilly.（第 8 章「The Trouble with Distributed Systems」對部分失敗的處理有通用參考價值）

Chapter 5 —多 Provider AI 路由：前瞻性容錯架構設計

任何依賴單一 AI 供應商的系統都是脆弱系統。多路徑容錯應該是架構起點，不是事故後的補丁。

目錄

- 5.1 單一供應商的結構性風險
- 5.2 modelRouter 架構
- 5.3 核心工程難題
- 5.4 重試策略疊加陷阱
- 5.5 切換觸發與觀察性
- 5.6 備援路徑的健康管理
- 5.7 函數骨架與資料結構
- 本章要點
- 參考資料

5.1 單一供應商的結構性風險

在討論「如何做容錯」之前，先說清楚**為什麼必須做**。依賴單一 AI 供應商存在五個**結構性**風險，無論供應商品牌大小都無法迴避：

1. **共用 token bucket** —所有客戶、所有模型共用同一個速率限制池，單一客戶的流量尖峰會擴散成全體退款時間
2. **模型下線 / 版本切換** —供應商退役舊模型時（如 `gpt-4-0314`、`claude-2`），所有依賴該模型的功能需即時切換
3. **區域故障** —單一雲端 region 故障會中斷該區域所有使用者
4. **帳戶 / 計費問題** —信用卡過期、扣款延遲、API key 輪轉錯誤都會造成完全斷線
5. **政策與地緣變動** —供應商調整服務範圍、某地區政策變化、模型使用條款改變

值得強調的是：**這五類風險不是「發生才處理」，而是「一定會發生」**。依賴任何單一供應商都等於把上述五種風險鎖進自己的服務 SLA。對 SaaS 而言這是不可接受的設計。

5.2 modelRouter 架構

百原 GEO 採用**主 / 備雙路徑**的抽象層，命名為 `modelRouter` 服務：

Fig 5-1：雙路徑路由架構

```
flowchart LR
    App[應用層 <br/>scan / eval / hallucination] --> MR[modelRouter]
    MR -->| 主路徑 | Agg[OpenAI-Compatible<br/> 聚合中轉]
    MR -->| 備援 | D1[OpenAI 直連]
    MR -->| 備援 | D2[Anthropic 直連]
    MR -->| 備援 | D3[Google 直連]
    MR -->| 備援 | D4[DeepSeek 直連]
    MR -->| 備援 | D5[Moonshot 直連]
    MR -->| 備援 | Dn[其他廠商直連]
    Agg --> LLM1[GPT-4o / Claude / Gemini / ...]
    D1 --> LLM1
```

Fig 5-1: 業務層呼叫抽象介面 *`modelRouter.complete()`*，底層路徑選擇對應用透明。主 / 備決策在 router 內部封裝。

兩條路徑的分工

路徑	特性	用途
主路徑（聚合中轉）	一個 API endpoint 涵蓋多家模型、計費統一、接入成本低	日常掃描、評分、一般推論
備援路徑（原生直連）	每家廠商獨立 API key、獨立計費、需各自 SDK	主路徑失敗時自動切換、或刻意需要廠商特定功能時

主路徑的優勢是**一次接入、處處可用**；備援路徑的優勢是**獨立性**。兩者之間透過 `modelRouter` 的**抽象介面**解耦，業務層完全不需要知道當前請求走哪條路徑。

5.3 核心工程難題

雙路徑聽起來簡單，實作時會遇到三個棘手的問題。

5.3.1 Model ID 在聚合層與原生層不一致

這是最常被低估的問題。同一個模型在聚合中轉與廠商原生 API 的識別字串常常不同：

廠商原生 ID	聚合中轉常見 ID
`deepseek-chat`	`deepseek-v3`
`moonshot-v1-8k`	`kimi-k2`
`claude-3-5-sonnet-20241022`	`claude-3-5-sonnet`
`qwen-plus`	`qwen3-plus`
`meta-llama/Llama-3.3-70B-Instruct`	`llama-3.3-70b`

不處理這個映射會導致：**主路徑成功的模型名稱，丟進直連 API 直接 404**。解法是維護一張對照表 `DIRECT_MODEL_ID_MAP`：

```
const DIRECT_MODEL_ID_MAP = {
  // aggregator ID → direct provider ID
  'deepseek-v3':          { provider: 'deepseek',  model: 'deepseek-chat' },
  'deepseek-r1':          { provider: 'deepseek',  model: 'deepseek-chat' },
  'kimi-k2':              { provider: 'moonshot',  model: 'moonshot-v1-8k' },
  'qwen3-plus':           { provider: 'alibaba',   model: 'qwen-plus' },
  'grok-2':               { provider: 'xai',       model: 'grok-2-latest' },
  'claude-3-5-sonnet':    { provider: 'anthropic', model: 'claude-3-5-sonnet-20241022' },
  // ... full mapping covers 15+ entries
};
```

每次新增或汰換模型都必須同步更新這張表，否則某個路徑會悄悄失效。

5.3.2 extraParams 差異

不同模型的原生 API 有各自的必要參數，聚合層通常會幫忙補上預設值；切到直連時這些參數需要自己給：

模型家族	需要的 extraParams
Qwen3 系列	`enable_thinking: false`（否則回傳推理過程，超長耗 token）
DeepSeek-R1	預設走推理模式，回應慢；需 `temperature: 0.6` 穩定輸出

模型家族	需要的 extraParams
Claude（Anthropic SDK）	`max_tokens` **必填**（OpenAI 系列可不給）
Gemini	`safetySettings` 預設太嚴，需放寬否則商業內容誤擋

實作上為每個 provider 維護一張 `DIRECT_EXTRA_PARAMS` 對照，在切換路徑時自動合併。

5.3.3 特殊推理模型的取捨

DeepSeek-R1 與 **OpenAI o1/o3** 屬於「推理型」模型，生成前會先做內部思考，回應時間往往 15–30 秒，超過我們掃描器的 25 秒 timeout。

解法不是延長 timeout（會拖垮整體掃描節奏），而是**在 fallback 時切換到同廠牌的非推理版本**：

主路徑模型	Fallback 直連模型	妥協
`deepseek-r1`	`deepseek-chat`	失去 reasoning_content 欄位
`o1-mini`	`gpt-4o-mini`	失去推理鏈，但 citation 偵測不依賴推理

對 GEO 掃描來說，**citation 偵測只需要最終文字**，推理鏈是加分不是必需。犧牲 reasoning_content 換取完成率是合理取捨。

5.4 重試策略疊加陷阱

這是實務上最常踩的坑。典型系統至少有三層重試：

```
Business Layer
                    withRetry(fn, { retries: 3 })
Router Layer
                    主路徑失敗切備援（= 又一次嘗試）
SDK Layer
                    OpenAI SDK 內建 maxRetries: 2
```

三層相乘，一次 OpenAI SDK timeout 30s × 2 retries × withRetry 3 次 × 主備切換 1 次 = **最壞狀況 7 分鐘**才確認失敗。掃描器預期一次請求在 25 秒內回結果，結果單一請求卡住整個 worker 超過十倍時間。

正確的設計原則

> 只讓最外層決定何時放棄。所有內層一律 `maxRetries: 0`、`timeout` 明確設置。

具體實作：

```
// SDK layer: no internal retry
const openai = new OpenAI({
  apiKey: API_KEY,
  maxRetries: 0,
  timeout: 20_000, // 20s hard limit
});

// Router layer: single failover, no loop
async function routeComplete(request) {
  try {
    return await primaryPath(request);
```

```
  } catch (err) {
    if (isRetryableError(err)) {
      return await fallbackPath(request);
    }
    throw err;
  }
}

// Business layer: caller decides retry policy
const result = await withRetry(
  () => modelRouter.complete(request),
  { retries: 2, backoff: 'exponential' }
);
```

三層各司其職，總耗時上限 = 20s × 2（主 + 備）× 3（外層重試）= 120s。可預測、可 monitor、可限制。

5.5 切換觸發與觀察性

並非所有錯誤都應該觸發切換。有些錯誤切換無用，反而浪費一次備援配額。

切換決策樹

```
flowchart TD
    E[Primary Path Error] --> C{Error Type}
    C -->|HTTP 5xx| S[切換到備援]
    C -->|HTTP 429 rate-limit| S
    C -->|Timeout| S
    C -->|Network Error| S
    C -->|HTTP 401 auth| X[直接拋錯 <br/> 切換也沒用]
    C -->|HTTP 400 param| X
    C -->|HTTP 402 payment| X
    C -->|Model not found| S
    S --> R[Record Event<br/>provider, model, reason, duration]
    X --> R
    R --> O[Emit metrics<br/>for observability]
```

Fig 5-2: 切換決策樹。5xx / 429 / timeout / 網路錯誤是可切換的「transient error」；401 / 400 / 402 是「permanent error」不該切換。

可觀察性必要欄位

每次切換事件必須記錄以下欄位，供後續分析與告警：

欄位	用途
`provider_primary`	主路徑提供者
`provider_fallback`	備援提供者（若有觸發）
`model_requested`	業務層請求的模型 ID
`model_actual`	實際使用的模型 ID（fallback 可能不同）
`reason`	transient / permanent / timeout / model_not_found 等
`latency_primary_ms`	主路徑耗時（包含 timeout 等待）
`latency_fallback_ms`	備援耗時
`status`	success_primary / success_fallback / both_failed

這些 metric 累積下來能回答重要問題：「哪個 provider 最不穩?」「哪個模型 fallback 觸發率最高?」「什麼時段最容易出事?」沒有這些資料，就只能靠客訴 debug。

5.6 備援路徑的健康管理

備援路徑最糟的狀態不是「失敗」，而是**「平常沒人用，真正需要時才發現壞掉」**。以下三個機制緩解此風險。

5.6.1 啟動時 Smoke Test

每次 worker 啟動時對**所有備援 provider** 發一次極簡請求（如 `"ping"` 或單一字 completion）：

```
async function smokeTestFallbacks() {
  const providers = Object.keys(DIRECT_CLIENTS);
  const results = await Promise.allSettled(
    providers.map(p => pingProvider(p, { timeout: 5_000 }))
  );
  return results.map((r, i) => ({
    provider: providers[i],
    ok: r.status === 'fulfilled',
    error: r.status === 'rejected' ? r.reason.message : null,
  }));
}
```

結果輸出到 startup log 與 `/health/fallbacks` endpoint。任何一條路徑失效，維運端能立刻知道。

5.6.2 週期性健康檢查

每 30 分鐘對備援做一次輕量檢查，用極低成本的 prompt（`"OK"` 回覆 2 token）。連續三次失敗則在 UI 的 provider 列表中標示「備援不可用」。

5.6.3 無法覆蓋的廠商

並非每個 provider 都有可用的備援。三種常見情況：

類型	範例	處理策略
閉源無公開 API	部分內部企業模型	UI 明示「無備援」、主路徑失敗即 fail
需客戶自備 API Key	部分付費 frontier model	讓客戶在設定頁自填 key，空著不自動 fallback
地緣政策限制	某些跨境服務	依客戶所在地區動態決定是否啟用

透明度比「假裝有覆蓋」更重要。告訴使用者「這個 provider 沒備援」，遠好過在出事時才發現。

5.7 函數骨架與資料結構

5.7.1 modelRouter 主入口

```
export async function complete({ prompt, model, temperature, ...opts }) {
  const request = buildRequest({ prompt, model, temperature, ...opts });

  try {
```

```
    const result = await callPrimary(request);
    emitMetric({ status: 'success_primary', model });
    return result;
  } catch (err) {
    if (!isRetryableError(err)) {
      emitMetric({ status: 'permanent_fail', reason: err.code, model });
      throw err;
    }

    const fallback = resolveFallback(model);
    if (!fallback) {
      emitMetric({ status: 'no_fallback_available', model });
      throw err;
    }

    const fallbackReq = mapRequestForProvider(request, fallback);
    try {
      const result = await callDirect(fallback.provider, fallbackReq);
      emitMetric({
        status: 'success_fallback',
        provider_fallback: fallback.provider,
        model_actual: fallback.model,
      });
      return result;
    } catch (fbErr) {
      emitMetric({ status: 'both_failed', model });
      throw fbErr;
    }
  }
}
```

5.7.2 resolveFallback

```
function resolveFallback(aggregatorModelId) {
  // 1. Exact match in DIRECT_MODEL_ID_MAP
  if (DIRECT_MODEL_ID_MAP[aggregatorModelId]) {
    return DIRECT_MODEL_ID_MAP[aggregatorModelId];
  }

  // 2. Pattern match for versioned models
  //    e.g. "deepseek-r1-0528" → "deepseek-chat"
  for (const [pattern, target] of DIRECT_PATTERN_MAP) {
    if (pattern.test(aggregatorModelId)) return target;
  }

  // 3. No fallback available
  return null;
}
```

這兩個函數是整個路由架構的核心；其他的 token counting、prompt assembly、response normalization 都建立在它們之上。

本章要點

- 單一供應商依賴是結構性風險，不是「有餘力再做」的選項
- `modelRouter` 以主 / 備雙路徑抽象，業務層對底層選擇完全無感
- 三個核心難題：Model ID 映射、extraParams 差異、推理模型 timeout 取捨
- 重試必須集中於最外層，SDK 與 router 層 `maxRetries: 0`，避免指數級耗時
- 切換決策依錯誤類型而定，transient 才切、permanent 直接拋
- 備援路徑需主動健康管理（smoke test + 週期檢查 + 不可覆蓋廠商的透明揭露）

參考資料

- Ch 2 —系統總覽
- Ch 4 —Stale Carry-Forward
- OpenAI. Node.js SDK retry configuration. https://github.com/openai/openai-node
- Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2nd ed.). Pragmatic Bookshelf. （Circuit Breaker、Bulkhead、Timeout 三個 pattern 的經典參考）

Chapter 6 —AXP 影子文檔：用 Cloudflare Worker 對 AI Bot 服務乾淨內容

給人類看的網站和給 AI 看的內容不該相同。強行用同一份 HTML 服務兩方，兩方都吃虧。

目錄

- 6.1 問題：為何同一份 HTML 服務兩方都吃虧
- 6.2 AXP 設計：影子文檔的結構
- 6.3 Cloudflare Worker 注入機制
- 6.4 AI Bot UA 清單與識別策略
- 6.5 SaaS 自家品牌的路徑衝突
- 6.6 Sitemap 自動產生
- 6.7 JSON-LD 扁平化的踩坑紀錄
- 6.8 GSC 索引血淚清單
- 6.9 RAG 知識庫串接：品牌專屬知識庫自動化
- 本章要點
- 參考資料

6.1 問題：為何同一份 HTML 服務兩方都吃虧

現代網站為人類設計，充滿：

- 客戶端渲染（CSR）—內容要等 JavaScript 跑完才出現
- 動態載入的圖卡、Carousel、Modal
- Cookie consent banner、廣告追蹤、A/B 測試 SDK
- 語意不明的 `<div class="col-md-6">` 嵌套幾十層
- 背景播放影片、WebGL、動畫

這些元素對**人類**是使用體驗，對 **AI 爬蟲**是噪音。AI Bot（GPTBot、ClaudeBot、Perplexity、Googlebot 等）抓取一個現代品牌頁面時，常見三種失敗：

1. **執行 JS 失敗或逾時** —多數 AI Bot 不執行 JS 或僅有受限執行，SPA 頁面只抓到一個 `<div id="app"></div>` 空殼
2. **抽取主體內容失敗** —HTML 噪音太多，AI 分不清哪段是品牌資訊、哪段是 UI chrome
3. **結構化資料缺失** —Schema.org JSON-LD 常被塞在動態生成的位置，AI 抓不到

結果是：AI 對品牌的認知要嘛**錯誤**、要嘛**稀薄**。而解法不是改造整個網站去遷就 AI，而是**為 AI 準備一份專屬的、乾淨的影子內容。**

Fig 6-1：同一品牌兩種視角

```
flowchart LR
    Brand[品牌資料] --> H[人類版網站 <br/>React / Vue / Next.js<br/> 完整 UI / 動畫 / 廣告]
    Brand --> A[AI 版影子文檔 <br/>Pure HTML + JSON-LD<br/> 純語意、無噪音]
    H -->|Browser| User[使用者體驗]
    A -->|AI Bot| LLM[模型訓練與檢索]
```

Fig 6-1: 同一份品牌資料導出兩種呈現：人類版優化體驗、AI 版優化語意。

6.2 AXP 設計：影子文檔的結構

AXP（AI-ready eXchange Page）是百原對這類影子文檔的命名。一個 AXP 頁面由三個部分組成：

Fig 6-2：AXP 文件三層結構

```
flowchart TB
    subgraph AXP["AXP 影子文檔"]
      HTML[" 純 HTML 語意骨架 <br/>h1 / h2 / p / ul / table"]
      JSONLD[" Schema.org JSON-LD<br/>Organization / Service / Person<br/> 三層 @id 互連"]
      MD[" Markdown 原文塊 <br/> 便於 RAG 切片與向量化"]
    end
    HTML --> AI[AI Bot Consumption]
    JSONLD --> AI
    MD --> AI
```

Fig 6-2: 三層齊備讓不同類型的 AI 爬蟲各取所需。純 HTML 給粗粒度抓取、JSON-LD 給知識圖譜、Markdown 給 RAG。

三層的實際樣貌

① 純 HTML 骨架

```html
<!DOCTYPE html>
<html lang="zh-TW">
<head>
  <meta charset="utf-8">
  <title>百原科技 —生成式引擎優化 SaaS</title>
  <meta name="description" content=" 百原科技是台灣首家 GEO SaaS...">
  <link rel="canonical" href="https://geo.baiyuan.io/">
</head>
<body>
  <main>
    <h1>百原科技</h1>
    <section>
      <h2>公司簡介</h2>
      <p>成立於 2024 年，專注於...</p>
    </section>
    <section>
      <h2>服務項目</h2>
      <ul>
        <li>GEO 掃描與評分</li>
        <li>AXP 影子文檔生成</li>
      </ul>
    </section>
  </main>
</body>
</html>
```

② Schema.org JSON-LD（詳見 Ch 7）

③ Markdown 原文塊（給 RAG 切片用）

```
# 百原科技

## 公司簡介
成立於 2024 年，專注於生成式引擎優化 SaaS 研發...
```

```
## 服務項目
- GEO 掃描與評分
- AXP 影子文檔生成
```

三層共存於同一個 URL 回應中：HTML 作為主體、JSON-LD 放於 `<script type="application/ld+json">`、Markdown 放於 `<script type="text/markdown" id="axp-markdown">`。

6.3 Cloudflare Worker 注入機制

AXP 的交付方式是**邊緣注入**：在 CDN 層級攔截請求，依據 User-Agent 決定回傳什麼內容。我們採用 Cloudflare Workers。

Fig 6-3：Worker 路由決策流程

```
flowchart TD
    Req[HTTP Request] --> UA{User-Agent<br/> 匹配 AI Bot?}
    UA -->| 是 | Cache{Worker Cache<br/> 命中?}
    UA -->| 否 | Pass[直接透傳原站 <br/> 客戶 origin]
    Cache -->| 命中 | Serve[回傳快取影子文檔]
    Cache -->| 未命中 | Fetch[從 geo.baiyuan.io API<br/> 拉取該品牌影子文檔]
    Fetch --> Store[寫入 Worker Cache<br/>TTL 15 分鐘]
    Store --> Serve
    Pass --> Origin[客戶 Origin Server]
```

Fig 6-3: Worker 先快取、未命中才回源拉 AXP；人類請求直接透傳，延遲與原站相同。

Worker pseudo code

```typescript
export default {
  async fetch(request, env) {
    const ua = request.headers.get('user-agent') || '';
    const url = new URL(request.url);

    // 1. 非 AI Bot：透傳原站
    if (!isAIBot(ua)) {
      return fetch(request); // proxy to customer origin
    }

    // 2. AI Bot：嘗試快取
    const cacheKey = `axp:${url.hostname}:${url.pathname}`;
    const cached = await env.KV.get(cacheKey);
    if (cached) {
      return new Response(cached, {
        headers: { 'content-type': 'text/html; charset=utf-8' },
      });
    }

    // 3. 未命中：回源拉 AXP
    const axpUrl = `https://api.geo.baiyuan.io/axp?host=${url.hostname}&path=${url.pathname}`;
    const axpRes = await fetch(axpUrl);
    if (!axpRes.ok) {
      return fetch(request); // 拉 AXP 失敗，fallback 原站
```

```
    }

    const body = await axpRes.text();
    await env.KV.put(cacheKey, body, { expirationTtl: 900 });
    return new Response(body, {
      headers: { 'content-type': 'text/html; charset=utf-8' },
    });
  },
};
```

關鍵設計點:

- AI Bot 請求與人類請求**走完全不同路徑**
- 任何拉 AXP 失敗都要能 fallback 原站，不能讓客戶的 AI 流量被 404
- Cache TTL 15 分鐘是「新鮮度 vs 後端壓力」的取捨

6.4 AI Bot UA 清單與識別策略

目前百原平台識別 **25 種** AI Bot UA，依功能分為四類:

Fig 6-4：AI Bot UA 分群

```
flowchart LR
    subgraph LLM[" 大型語言模型爬蟲 (10)"]
      GPTBot
      ClaudeBot
      GoogleExtended[Google-Extended]
      CCBot[CCBot / Common Crawl]
      MetaBot[FacebookBot / Meta-ExternalAgent]
      ByteBot[Bytespider]
      AnthropicBot[anthropic-ai]
      AppleBot[Applebot-Extended]
    end
    subgraph Search[" 搜尋型爬蟲 (6)"]
      Perplexity[PerplexityBot]
      ChatGPTUser[ChatGPT-User]
      PerplexityUser[Perplexity-User]
      YouBot[YouBot]
      PhindBot
    end
    subgraph Preview["Preview/Link 生成 (5)"]
      LinkedInBot
      FacebookExt[facebookexternalhit]
      TwitterBot[Twitterbot]
      DiscordBot[Discordbot]
    end
    subgraph RAG["RAG / 企業爬蟲 (4)"]
      CohereBot[cohere-ai]
      DiffBot[Diffbot]
      OmigoBot[Omigo]
    end
```

Fig 6-4: 25 種 AI Bot 的分群示意。百原平台對四類全部啟用 AXP 注入，但可依客戶需求於 admin 設定分群停用。

識別策略

實作上使用**正則合併**而非條列 if-else，以維護性優先：

```
const AI_BOT_REGEX = new RegExp(
  [
    'GPTBot', 'ChatGPT-User', 'OAI-SearchBot',
    'ClaudeBot', 'anthropic-ai', 'Claude-Web',
    'Google-Extended', 'GoogleOther',
    'PerplexityBot', 'Perplexity-User',
    'CCBot', 'Bytespider', 'FacebookBot',
    'Meta-ExternalAgent', 'Applebot-Extended',
    'cohere-ai', 'Diffbot', 'YouBot', 'PhindBot',
    'LinkedInBot', 'facebookexternalhit', 'Twitterbot',
    'Discordbot', 'Omigo', 'DuckAssistBot',
  ].join('|'),
  'i'
);

function isAIBot(ua) {
  return AI_BOT_REGEX.test(ua);
}
```

UA 清單每季更新一次，新出現的爬蟲（如 `OAI-SearchBot` 在 2025 年 7 月首次出現）需即時補入。

6.5 SaaS 自家品牌的路徑衝突

一個實務上的特殊案例：**當 SaaS 平台自己也是該 SaaS 的使用者時**（dogfooding），自家域名既要服務「平台使用者」（登入後用產品）又要服務「品牌官網訪客」（匿名讀介紹）。

百原自家的 `geo.baiyuan.io` 就是這個情境：

路徑	人類使用者	AI Bot
`/`	行銷首頁（公開）	AXP「百原科技」品牌頁
`/dashboard`	登入後儀表板（私有）	403，不該被 AXP 化
`/features`, `/pricing`	產品介紹（公開）	AXP 對應的 service 頁
`/login`, `/signup`	登入 / 註冊（公開但無品牌資訊）	不注入，透傳

決策樹

```
flowchart TD
    R[Request] --> B{AI Bot?}
    B -->| 否 | P1[透傳原站]
    B -->| 是 | P{路徑分類}
    P -->| 行銷 / 品牌頁 | A[注入 AXP]
    P -->| 登入後功能 | X1[回 403 Forbidden]
    P -->| 無品牌內容的公開頁 | X2[透傳原站]
```

Fig 6-5: 路徑分類表由 admin 在每個品牌的設定頁維護；不在表中的新路徑預設透傳原站，採保守策略。

6.6 Sitemap 自動產生

AI Bot 爬取效率依賴 `sitemap.xml`。AXP 模式下必須**動態產生與 AXP 路徑完全對齊的 sitemap**，否則會出現「sitemap 列了某 URL，但 Worker 該路徑沒注入 AXP」的混亂。

百原平台為每個客戶域名自動產生 sitemap，規則：

- 以該品牌的 `brand_locations`、`brand_services`、`brand_employees` 表動態生成 URL
- 每個 URL 註記 `<lastmod>` 為對應實體的 `updated_at`
- `<priority>` 依路徑類型：首頁 1.0、服務頁 0.8、員工頁 0.6
- robots.txt 主動聲明 `Sitemap: https://<domain>/sitemap.xml`

Sitemap 與 AXP 同樣由 CF Worker 注入；人類用戶若輸入 `/sitemap.xml` 也會看到（這是 SEO 通用慣例，不必隱藏）。

6.7 JSON-LD 扁平化的踩坑紀錄

Schema.org 規範允許**嵌套陣列**（nested array），但實務上會踩到以下問題：

Fig 6-6：錯誤與正確範例對比

```
//  錯誤：nested array（部分 AI 解析器會整塊拒絕）
{
  "@context": "https://schema.org",
  "@graph": [
    [
      { "@type": "Organization", "name": " 百原科技" }
    ],
    [
      { "@type": "Service", "name": "GEO 掃描" }
    ]
  ]
}

//  正確：flat array
{
  "@context": "https://schema.org",
  "@graph": [
    { "@type": "Organization", "@id": "#org", "name": " 百原科技" },
    { "@type": "Service", "@id": "#svc-scan", "name": "GEO 掃描",
      "provider": { "@id": "#org" } }
  ]
}
```

Fig 6-6: 實體間關聯用 *@id* 引用，而非用陣列嵌套表達。此為 Schema.org 工具驗證的硬性要求。

維持 flat 化 + 用 `@id` 連結的好處：

- Google Rich Results 工具能驗證通過
- Wikidata / Wikipedia 的結構化資料抽取器可對齊
- AI 的知識圖譜建構更穩定

6.8 GSC 索引血淚清單

從 2024 年到 2025 年的實作過程中，我們在 Google Search Console（GSC）索引上踩過的坑：

踩坑	症狀	根因	解法
`noindex` meta 誤覆蓋	GSC 顯示「已被 `noindex` 排除」	UAT 環境 `.env` 被誤部到 PROD	環境變數加 `strict` 檢查，啟動時拒絕不合理組合
canonical 跨域	PROD 頁面 canonical 指向 UAT 域名	同一 code base 兩環境共用 canonical 邏輯	canonical 改為動態依 `request.hostname` 產生
Bot UA 漏列	GSC 指數變動但某些 AI 引用消失	新型 Bot 未加入 UA regex	每季檢視 CF Worker log 未命中 UA
Sitemap 不一致	GSC 報 `Discovered – currently not indexed`	AXP 頁存在但 sitemap 遺漏	Sitemap 生成改為從同一來源（AXP 索引表）推導
HTTPS/HTTP 混用	robots.txt 在 HTTP 是 200、HTTPS 是 404	Worker 未處理 `http://` 流量	強制 301 → HTTPS，且 robots 同步注入

這些坑不是 AXP 特有，但 **AXP 放大了它們的嚴重性**—因為 AI Bot 的抓取重試頻率低於 Googlebot，一次錯誤可能要等幾週才有機會被再次抓取。寧可先部署一個 `check-prod-seo.sh` 腳本在 CI 檢查上述五類問題，也不要上線後才發現。

6.9 RAG 知識庫串接：品牌專屬知識庫自動化

AXP 頁面的品質不只取決於「能否服務 AI Bot」，更取決於**內容本身有多少品牌知識**。純由 LLM 憑空生成的頁面容易出現幻覺或空泛描述；若能在生成時注入來自品牌自身知識庫的事實，輸出品質會有量級的差距。

6.9.1 品牌隔離：每個品牌一座獨立知識庫

百原採用「中央共用 RAG 引擎」（§9.4），但每個品牌的文件存於**獨立的 Knowledge Base（KB）**，以 `rag_kb_id` 區隔查詢範圍：

- 同一租戶下的多品牌（如「醫美品牌」與「美食品牌」）知識互不干擾
- AXP 生成時指定 `kbId` 查詢，確保回傳的是該品牌自身的事實
- 品牌刪除時可一併清除對應 KB，無遺留資料風險

```
flowchart LR
    subgraph Tenant[" 同一租戶"]
      BrandA[" 品牌 A<br/>rag_kb_id: kb-aaa"]
      BrandB[" 品牌 B<br/>rag_kb_id: kb-bbb"]
    end
    subgraph RAG[" 中央 RAG 引擎"]
      KBA["KB: kb-aaa<br/> (品牌 A 專屬) "]
      KBB["KB: kb-bbb<br/> (品牌 B 專屬) "]
    end
    BrandA -->|askWithKbId| KBA
    BrandB -->|askWithKbId| KBB
```

Fig 6-7: 品牌層級 KB 隔離。中央引擎共用，知識互不污染。

6.9.2 seedBrandRAGKB：AXP 啟用時自動建立並填充知識庫

新品牌啟用 AXP（enableAXP API）時，系統在背景非同步執行三步驟：

1. **建立 KB**（若不存在）—ragCreateKnowledgeBase 返回新 kbId，寫入 brand_rag_configs
2. **上傳品牌 Profile**（文字文件）—品牌名稱、行業、簡介、官網、核心關鍵字，組合成結構化文字作為 KB 的錨點知識
3. **上傳官網頁面 URL**（最多 20 筆，依 geo_importance 降序）—RAG 後端爬取並向量化，補充動態網站內容

```
// enableAXP 的 setImmediate block 中（非阻塞）
await initialCrawl(brandId, tenantId);   // 生成 AXP 頁面
await seedBrandRAGKB(brandId, queryFn);  // 平行 seed RAG KB
```

關鍵設計考量：- **非阻塞**—seed 過程不影響 AXP 頁面的即時回應 - **冪等**—重複呼叫只補充新文件，不產生重複 KB - **Fallback** —任何 URL 爬取失敗都 gracefully skip，不影響其他 URL

6.9.3 關鍵字注入：讓 AXP 內容有品牌聲音

brand.keywords（品牌設定的目標 GEO 關鍵字）以 keywordsHint 的形式注入每個 RAG 查詢提示：

```
// hybridCoordinator.service.js
const keywordsHint = keywords.length
  ? `\n\n請在回答中自然地融入以下目標關鍵字（不強求每個都出現）：${keywords.join('、')}`
  : '';
const question = PAGE_TYPE_QUESTIONS[pageType](brandName) + keywordsHint;
```

定價頁（pricing_summary）與功能頁（product_features）特別容易出現「純表格、零關鍵字」，因此 RAG 提示強制要求先生成**關鍵字豐富的開場簡介段落**，再接表格內容：

1. 開場簡介段落（2-3 句）：說明品牌定位，並自然融入目標關鍵字
2. 完整定價表格：只列知識庫確認的資料，不得推測或捏造

實測關鍵字覆蓋率（ILIKE 子字串匹配，5 個品牌 × 6 頁型，2026-04-21）：

品牌	關鍵字數	最低覆蓋頁型	最低覆蓋率	多數頁型覆蓋
百原 GEO	13	pricing_summary	9/13	11–13/13
百原 RAG 知識庫	12	pricing_summary	7/12	11–12/12
愛上生活	10	pricing_summary	7/10	9–10/10
PIF	12	faq / pricing	7/12	10–12/12
百原科技有限公司	10	pricing_summary	1/10 ★	9–10/10

★ 品牌官網無公開定價頁，RAG 正確拒絕推測，此為預期行為。

6.9.4 content_preview：每頁唯一的品質監控信號

AXP 頁面列表 API 新增 content_preview 欄位：取 content_md 前 150 字元（去除多行 HTML 注釋後）作為頁面卡片摘要。

這解決了一個實務問題：同一品牌 6 個頁型若全部顯示相同的「品牌指紋短語」（fingerprint_phrase），後台使用者無法確認各頁是否正確生成。

```
-- 去除多行 HTML 注釋，取前 150 字元
LEFT(REGEXP_REPLACE(content_md, '<!--[\s\S]*?-->', '', 'g'), 150) AS content_preview
```

注意 regex 需使用 [\s\S]*?（dotall 模式）而非 [^>]*，否則跨行 HTML 注釋無法正確去除。

本章要點

- 同一份 HTML 難以同時服務人類體驗與 AI 爬蟲，AXP 是解耦的必要設計
- AXP 三層結構：純 HTML 骨架 + Schema.org JSON-LD + Markdown 原文塊
- Cloudflare Worker 邊緣偵測 UA，AI Bot 請求與人類請求走完全不同路徑
- 25 種 AI Bot UA 以正則合併維護，每季檢視新型爬蟲
- SaaS 自家品牌的路徑衝突需於 admin 維護分類表；預設保守透傳
- Sitemap 動態產生、與 AXP 路徑對齊；JSON-LD 採 `@id` flat 化而非 nested array
- GSC 索引問題被 AI 爬蟲放大，需 pre-flight 腳本把關
- 每個品牌有獨立 RAG 知識庫（`rag_kb_id`）；`seedBrandRAGKB` 在 AXP 啟用時自動上傳品牌 Profile 與官網頁面 URL，無需人工干預
- `brand.keywords` 以 `keywordsHint` 注入 RAG 查詢，定價頁與功能頁強制生成關鍵字豐富的開場段落；`content_preview` 取代 `fingerprint_phrase` 作頁面品質監控

參考資料

- Ch 7 —Schema.org Phase 1：25 產業 × 三層 @id
- Ch 8 —GBP API 整合策略
- Cloudflare. Workers Runtime Documentation. https://developers.cloudflare.com/workers/
- OpenAI. GPTBot User Agent. https://platform.openai.com/docs/gptbot
- Anthropic. ClaudeBot documentation. https://support.anthropic.com/en/articles/8896518
- Google. Google-Extended and generative AI training. https://developers.google.com/search/docs/crawling-indexing/overview-google-crawlers

---

Chapter 7 —Schema.org Phase 1：25 產業 × 三層 @id 互連

Schema.org 不是「加幾個 tag」這麼簡單。沒有產業特化、沒有實體互連、沒有自動生成的結構化資料，對 AI 而言幾乎等於沒有。

目錄

- 7.1 Schema.org 在 AI 時代的角色重定位
- 7.2 25 類產業特化 @type 的設計
- 7.3 三層 @id 互連知識圖
- 7.4 實體商家 vs 線上服務：欄位權重的分歧
- 7.5 資料完整度演算法
- 7.6 Wizard + Edit 雙入口設計
- 7.7 GBP URL Parser
- 7.8 函數骨架
- 本章要點
- 參考資料

7.1 Schema.org 在 AI 時代的角色重定位

Schema.org 誕生於 2011 年，是 Google、Bing、Yahoo、Yandex 聯合推動的結構化資料詞彙表[4]。它原本的主要用途是讓**傳統搜尋引擎**產出 Rich Results（評分星級、麵包屑、FAQ 展開等）。

2024 年之後，Schema.org 的角色出現兩個根本性轉變：

1. **從「搜尋引擎裝飾物」變成「AI 訓練語料結構化來源」**—主流大型語言模型預訓練時會消化 Common Crawl，Schema.org JSON-LD 是其中最密集的實體資料來源
2. **從「加分項」變成「必要項」**—沒有 Schema.org 的網站在 AI 眼中是「一團文字」；有 Schema.org 的網站是「一個可識別的實體」。差別等同於「圖片有沒有 alt」對視障使用者

本書把 Schema.org 視為**百原 GEO 優化路徑的第一個槓桿**。一個品牌若沒有完善的 Schema.org 結構，無論其他維度如何優化，AI 認知都難以穩定。

7.2 25 類產業特化 @type 的設計

Schema.org 規範中有數百個 `@type`，許多特化程度極高（例：`MedicalClinic`、`VeterinaryCare`、`CafeOrCoffeeShop`）。**對 AI 而言，選錯 @type 等於把自己塞到錯的抽屜**——AI 以 @type 為關鍵維度在知識圖譜中定位實體。

百原平台將常見產業歸納為 **25 類**，每類對應一組 Schema.org @type：

Fig 7-1：25 產業分類表（實體 16 + 線上 7 + 保底 2）

code	中文名稱	對應 Schema.org @type
`medical_clinic`	醫美診所	`MedicalClinic`, `LocalBusiness`
`dental_clinic`	牙科診所	`Dentist`, `LocalBusiness`
`general_clinic`	一般診所	`MedicalOrganization`, `LocalBusiness`
`beauty_salon`	美髮/美容	`BeautySalon`, `LocalBusiness`
`fitness`	健身房/瑜珈	`HealthClub`, `SportsActivityLocation`

[4] Schema.org. Schema.org Vocabulary Specification. https://schema.org/docs/schemas.html

code	中文名稱	對應 Schema.org @type
restaurant	餐廳	Restaurant, FoodEstablishment
cafe	咖啡店	CafeOrCoffeeShop
legal_service	律師事務所	LegalService, ProfessionalService
accounting	會計師事務所	AccountingService, ProfessionalService
real_estate	不動產仲介	RealEstateAgent, ProfessionalService
auto_repair	汽車維修	AutoRepair, AutomotiveBusiness
education_offline	補習班/培訓	EducationalOrganization, LocalBusiness
veterinary	寵物醫院	VeterinaryCare, MedicalOrganization
lodging	旅館/民宿	LodgingBusiness, Hotel
retail_store	零售店	Store, LocalBusiness
financial_service	金融服務	FinancialService, ProfessionalService
saas_application	SaaS 軟體	SoftwareApplication, Organization
web_application	網頁工具	WebApplication, Organization
mobile_app	手機 App	MobileApplication, Organization
ecommerce	純電商	OnlineStore, Organization
online_education	線上課程平台	EducationalOrganization
news_media	媒體/內容網站	NewsMediaOrganization
online_professional	線上顧問	ProfessionalService, Organization
other_physical	其他實體商家	LocalBusiness
other_online	其他線上服務	Organization

Fig 7-1: 實體 16、線上 7、保底 2。每類同時指定兩個 @type（主 + 副），以符合 Schema.org 允許 @type 為陣列的規範。

為何歸類為 25 類而非更細

Schema.org 規範中細分類型可達數百，但**過細的分類反而降低 AI 的識別率**。原因：

- AI 模型訓練時對**常見類型**的權重更高（例如 `Restaurant` 比 `FastFoodRestaurant` 更被識別）
- 客戶填寫時**選項過多會放棄**，25 類已是「覆蓋度」與「可用性」的平衡
- 特化細分的 @type 會在主 @type 之外作為 additional type 呈現，不強制所有客戶選擇

7.3 三層 @id 互連知識圖

Fig 7-2：三層實體知識圖

```
flowchart TB
    subgraph L1["Layer 1: 主體"]
      Org["Organization / LocalBusiness<br/>@id = #org<br/>name / description / url / logo /<br/>address
    end
    subgraph L2["Layer 2: 服務"]
      Svc1["Service<br/>@id = #svc-1"]
      Svc2["Service<br/>@id = #svc-2"]
      SvcN["..."]
    end
    subgraph L3["Layer 3: 人員"]
      Emp1["Person / Physician<br/>@id = #emp-1"]
      Emp2["Person / Attorney<br/>@id = #emp-2"]
    end
    Svc1 -->|provider| Org
```

```
    Svc2 -->|provider| Org
    Emp1 -->|worksFor| Org
    Emp2 -->|worksFor| Org
    Svc1 -->|performer| Emp1
    Svc2 -->|performer| Emp2
    Org -.->|sameAs| Wiki[Wikipedia]
    Org -.->|sameAs| WD[Wikidata]
    Org -.->|sameAs| LI[LinkedIn]
    Org -.->|sameAs| GBP[Google Business Profile]
```

Fig 7-2: 三層用 *@id* 互相引用形成知識圖譜；外部權威平台透過 *sameAs* 建立跨知識庫連結。

為何分三層而非一個 blob

實務上常見的錯誤是把所有資訊擠進一個 `Organization` 物件：

```
{
  "@type": "Organization",
  "name": " 某醫美診所",
  "employees": [
    { "name": " 林醫師", "jobTitle": " 院長" }
  ],
  "services": [
    " 電波拉皮", " 雙眼皮手術"
  ]
}
```

這種寫法的問題是：AI 無法把「林醫師」當成可被獨立引用的實體（Person entity）；「電波拉皮」只是字串不是服務實體（Service entity）。結果是「關於林醫師的問題」無法對應到任何結構化資料。

三層 @id 的寫法則建立可定址的實體：

```
{
  "@context": "https://schema.org",
  "@graph": [
    {
      "@type": ["MedicalClinic", "LocalBusiness"],
      "@id": "https://example.clinic/#org",
      "name": " 某醫美診所",
      "sameAs": [
        "https://www.wikidata.org/wiki/Q...",
        "https://www.linkedin.com/company/..."
      ]
    },
    {
      "@type": "Physician",
      "@id": "https://example.clinic/#emp-1",
      "name": " 林醫師",
      "jobTitle": " 院長",
      "worksFor": { "@id": "https://example.clinic/#org" }
    },
    {
      "@type": "Service",
      "@id": "https://example.clinic/#svc-thermage",
      "name": " 電波拉皮",
      "provider": { "@id": "https://example.clinic/#org" },
```

```
      "performer": { "@id": "https://example.clinic/#emp-1" }
    }
  ]
}
```

當使用者問 AI「誰做電波拉皮」時，AI 的推理路徑有完整的實體鏈可走，而不是從模糊字串匹配。

7.4 實體商家 vs 線上服務：欄位權重的分歧

`is_physical` 旗標決定了欄位完整度的權重表。兩者對 AI 引用率的影響維度完全不同：

Fig 7-3：欄位權重分歧

```
flowchart LR
    subgraph Physical[" 實體商家 (is_physical=true)"]
      P1[" 地址 15%"]
      P2["GBP Place ID 15%"]
      P3[" 營業時段 10%"]
      P4[" 電話 10%"]
      P5[" 服務項目 10%"]
      P6[" 員工名單 10%"]
      P7["url / name / logo 30%"]
    end
    subgraph Online[" 線上服務 (is_physical=false)"]
      O1["url 20%"]
      O2["description 15%"]
      O3["logo_url 10%"]
      O4[" 服務/產品功能 20%"]
      O5["sameAs 外部連結 15%"]
      O6["FAQ 10%"]
      O7[" 其他 10%"]
    end
```

Fig 7-3: 實體商家的「地址 + GBP」佔 30%，線上服務的「url + description」佔 35%。同一套演算法兩套權重，以正確反映使用者查詢 AI 的主要意圖類型。

設計依據

- **實體商家**：使用者問 AI 時往往帶地域（「台北信義區哪家醫美評價好」），AI 需要從 Schema.org 抽出地址與營業資訊；缺了這些，AI 回答無法落地
- **線上服務**：使用者問的是「功能型」問題（「最好的 CRM 是哪個」），AI 需要的是描述、差異化、同類比較；地址反而無關緊要

百原平台的 UI 依 `is_physical` 動態顯示 / 隱藏欄位：實體類客戶會看到「地址」「營業時段」卡片，線上類則不會；這是 **Ch 2** 提及的「Visibility Module」的具體體現。

7.5 資料完整度演算法

每個欄位有一個**權重**（0–100），填寫即加分。總完整度是各欄位權重的加權平均。核心計算邏輯：

```
function computeCompletion(brand, industry) {
  const weights = industry.is_physical ? PHYSICAL_WEIGHTS : ONLINE_WEIGHTS;
```

```
  let score = 0;
  let maxScore = 0;

  for (const [field, weight] of Object.entries(weights)) {
    maxScore += weight;
    if (isFilledMeaningfully(brand, field)) {
      score += weight;
    }
  }

  return Math.round((score / maxScore) * 100);
}

// 不只看欄位是否非空，還檢查「是否有意義」
function isFilledMeaningfully(brand, field) {
  const value = getField(brand, field);
  if (!value) return false;
  // 過濾佔位符
  if (typeof value === 'string' && PLACEHOLDER_PATTERNS.test(value)) return false;
  // 關聯表需至少一筆
  if (Array.isArray(value) && value.length === 0) return false;
  return true;
}
```

為何不只檢查「非空」

早期實作只判斷欄位是否非空，導致客戶把 `url` 填成 `"https://"`、`description` 填成 `" 公司"` 等佔位符來刷分。`isFilledMeaningfully` 追加三個檢查：

1. **佔位符正則** —`^(https?:\/\/)?$`、`^[a-zA-Z ]{1,3}$`、空白字元等
2. **最小長度門檻** —例如 description 至少 20 字才計入
3. **格式驗證** —URL 需可解析、電話需符合 E.164 等

這類「形式完整但實質無用」的案例很常見，尤其是客戶不熟悉 GEO 工具時。UI 不阻止填寫，但演算法不計入分數，避免誤導後續優化建議。

7.6 Wizard + Edit 雙入口設計

Fig 7-4：入口流程

```
flowchart TD
    Start{使用者類型} -->| 新建品牌 | Wiz[Wizard<br/> 線性 7 步驟]
    Start -->| 既有品牌 | Dash[Dashboard<br/> 完整度 Banner]
    Wiz --> W1[Step 1: 基本資訊]
    W1 --> W2[Step 2: 產業與描述]
    W2 --> W3[Step 3: 地址 位置 <br/> 僅 is_physical]
    W3 --> W4[Step 4: 營業時段 <br/> 僅 is_physical]
    W4 --> W5[Step 5: 服務項目]
    W5 --> W6[Step 6: 員工名單]
    W6 --> W7[Step 7: FAQ 社群]
    W7 --> Done[完成]
    Dash -->|<80%| Alert[紅/琥珀警示]
    Dash --> Edit[/brands/id/entity<br/> 自由跳轉任意 Card]
```

```
    Alert --> Edit
    Edit --> Save[儲存即時更新 <br/> 完整度 %]
```

Fig 7-4: 新品牌走 Wizard 保證首次覆蓋率；既有品牌走 Edit 自由更新。兩條路徑共用同一套 Card 元件 (DRY 原則)。

為何 Wizard 不強制所有欄位必填

設計上 Wizard 每一步都允許「暫時跳過」:

- **填寫疲勞**會讓使用者放棄整個流程；與其一次性要 100%，不如先讓他們拿到 60%
- **引導型 UI** 比**強制型 UI** 對使用者友善，符合 progressive disclosure 原則
- Wizard 結束後 Dashboard 會持續以 Banner 提醒未完成的欄位，形成**補完的二次機會**

這是產品哲學的選擇：**讓品牌先存在於 AI，再追求完美。**

7.7 GBP URL Parser

Google Business Profile（GBP）提供的地點識別有三種 ID 型別，客戶常常只拿到其中一種的 URL：

ID 類型	範例 URL	用途
`place_id`	`https://www.google.com/maps/place/?q=place_id:ChIJ...`	Place Details API 的 primary key
`FTID`	`https://maps.google.com/maps?ftid=0x0:0xe6...`	Google Maps 內部識別
`CID`	`https://www.google.com/maps?cid=...`	Customer ID，短網址形式

Fig 7-5：GBP URL Parser 決策樹

```
flowchart TD
    In[貼入任意 GMB URL] --> Split{URL 型別}
    Split -->| 含 place_id:| P[抽 place_id]
    Split -->| 含 ftid=| F[抽 FTID<br/> 轉換為 place_id]
    Split -->| 含 cid=| C[抽 CID<br/>Places API 查詢 → place_id]
    Split -->| 短網址 goo.gl| R[resolve 301 → 重新進入 Split]
    Split -->| 其他 | X[回傳 null<br/> 要求使用者貼完整 URL]
    P --> Done[回傳 Place ID]
    F --> Done
    C --> Done
```

Fig 7-5: parser 對四種 URL 格式各有分支；任何無法解析的 URL 回傳明確錯誤，不猜測。

為何 CID 需要查 API

CID 是 Google 內部流水號，無法直接轉換為 Place ID。parser 呼叫 Google Places API `findPlaceFromText` 用 CID 反查:

```
async function cidToPlaceId(cid) {
  const res = await fetch(
    `https://maps.googleapis.com/maps/api/place/findplacefromtext/json?` +
    `input=cid:${cid}&inputtype=textquery&fields=place_id&key=${API_KEY}`
  );
```

```
  const data = await res.json();
  return data.candidates?.[0]?.place_id ?? null;
}
```

這個呼叫會計入 Google API 配額；parser 對同一 URL 設 24 小時 cache，避免重複消耗。

7.8 函數骨架

generateBrandEntitySchema

```
function generateBrandEntitySchema(brand, industry) {
  const base = `https://${brand.primary_domain}`;
  const graph = [];

  // Layer 1: Organization / LocalBusiness
  graph.push({
    '@type': industry.schema_types, // 陣列，如 ["MedicalClinic", "LocalBusiness"]
    '@id': `${base}/#org`,
    name: brand.name,
    url: brand.url,
    description: brand.description,
    logo: brand.logo_url,
    ...(industry.is_physical && {
      address: buildAddress(brand.location),
      telephone: brand.location?.telephone,
      openingHoursSpecification: buildHours(brand.hours),
      geo: buildGeo(brand.location),
    }),
    sameAs: buildSameAs(brand), // Wikipedia / Wikidata / LinkedIn / GBP
  });

  // Layer 2: Services
  for (const svc of brand.services ?? []) {
    graph.push({
      '@type': 'Service',
      '@id': `${base}/#svc-${svc.slug}`,
      name: svc.name,
      description: svc.description,
      provider: { '@id': `${base}/#org` },
    });
  }

  // Layer 3: Employees
  for (const emp of brand.employees ?? []) {
    graph.push({
      '@type': emp.specialized_type ?? 'Person', // Physician / Attorney / ...
      '@id': `${base}/#emp-${emp.slug}`,
      name: emp.name,
      jobTitle: emp.job_title,
      worksFor: { '@id': `${base}/#org` },
    });
  }
```

```
  return {
    '@context': 'https://schema.org',
    '@graph': graph,
  };
}
```

此函數是 AXP 生成流程（Ch 6）與 Closed-Loop 幻覺修復（Ch 9）的共同底層。

本章要點

- Schema.org 在 AI 時代從「搜尋引擎裝飾物」升級為「AI 訓練語料結構化來源」
- 25 類產業 enum（16 實體 + 7 線上 + 2 保底）覆蓋常見需求，同時保持可選性
- 三層 @id 互連（Organization / Service / Person）把 blob 資料變為可定址實體
- `is_physical` 旗標觸發兩套不同權重表；實體商家重「地址/GBP」、線上服務重「url/description」
- 完整度演算法過濾佔位符與形式化填寫，避免分數通膨
- Wizard 引導新品牌、Edit 服務既有品牌；DRY 共用同一套 Card 元件
- GBP URL Parser 支援 place_id / FTID / CID 三種格式

參考資料

- Ch 6 —AXP 影子文檔
- Ch 8 —GBP API 整合
- Ch 9 —Closed-Loop 幻覺修復

Chapter 8 —GBP API 整合策略：從事實主控源到對外輸出的單向同步

> 實體商家的資訊散落在網站、Facebook、LINE 官方帳號、GBP、平台 profile 之間。若沒有單一事實源，AI 看到的一定是拼貼與矛盾。

目錄

- 8.1 為何 GBP 是實體商家的事實主控源
- 8.2 GBP API 申請門檻與流程
- 8.3 代管模式選擇
- 8.4 欄位對應表
- 8.5 同步頻率與配額
- 8.6 無 Webhook 的補救方案
- 8.7 Phase 1–4 Roadmap
- 本章要點
- 參考資料

8.1 為何 GBP 是實體商家的事實主控源

對實體商家而言，Google Business Profile（GBP）是最權威的實體識別節點，原因有三：

1. **Google 搜尋 + Maps 使用者面**：查詢「台北大安區牙醫」時，GBP 的地點卡片往往是第一個被點擊的結果
2. **AI 訓練資料管道**：Google AI Overview、Perplexity、ChatGPT 等在處理地域查詢時，大量參考 GBP 的商家資訊（直接或間接）
3. **跨 Google 產品一致性**：Google Maps、Search、Assistant、AI Overview 全部共用同一份 GBP 資料

Fig 8-1：資料流：GBP → Schema.org → AXP → AI

```
flowchart LR
    GBP[(GBP<br/> 事實主控源)] -->|API 讀取 | Sync[百原同步器]
    Sync -->| 正規化 | DB[(百原 DB<br/>brand_*)]
    DB -->| 產生 | JSONLD[Schema.org JSON-LD]
    JSONLD --> AXP[AXP 影子文檔]
    AXP -->|CF Worker 注入 | AI[AI Bot]
    AXP -->|sitemap.xml| GB[Googlebot]
```

Fig 8-1: 資料單向流動：GBP 改 → 百原 DB 跟著改 → JSON-LD 重生 → AXP 更新 → AI 抓取。客戶只要在一處（GBP）維護資訊。

關鍵原則：**同步是單向的**。百原平台**不寫回** GBP（至少在 Phase 1–2），避免雙寫衝突與覆蓋風險。Phase 3 才開放 LocalPosts 寫入（見 §8.7）。

8.2 GBP API 申請門檻與流程

Google Business Profile API 不是公開註冊即可使用，而是需要申請核准。申請流程中幾個值得紀錄的關卡：

階段	內容	時程	備註
1. 前置條件	擁有 Google Cloud 專案 + 至少一個已驗證的 GBP	1 天	若無既有驗證商家，可用代管品牌的 GBP
2. 提交申請表	Use Case 說明、預估 QPM、資料用途	半天	Use Case 需聚焦在「為商家主提供管理工具」類型
3. Google 審核	Google 團隊評估申請適格性	7–10 個工作天	官方公布時程，實際可能更長
4. 核准啟用	取得 API 配額、開通相關 scope	即時	預設 QPM 300，可再申請提升

常見申請被拒的原因

- Use Case 模糊（「做 SEO 工具」被視為邊緣用途）
- 沒有已驗證商家（申請者必須有管理權）
- 提交表格誤入「公開論壇」而非正式管道（這是官方文件結構易混淆的坑）

百原的策略是：**在 Phase 1（Schema.org 手動填寫）階段就累積已驗證的 GBP 品牌**，Phase 2 申請時一次提交 5 個驗證案例作為適格性證明。

8.3 代管模式選擇

客戶的 GBP 要交給百原平台讀取，有兩種模式可選：

Fig 8-2：兩種代管模式對比

```
flowchart TB
    subgraph Manager[" 模式 A: Manager 加入"]
      C1[客戶於 GBP 介面 <br/> 加入百原為 Manager]
      C1 --> S1[百原以 Service Account<br/> 讀取該 Location]
      S1 --> Pro1[" 一次設定、長期有效 <br/> 需客戶會操作 GBP UI<br/> 百原可見所有 Location 資料"]
    end
    subgraph OAuth[" 模式 B: OAuth 授權"]
      C2[客戶於百原介面 <br/> 點擊「連結 Google」]
      C2 --> S2[OAuth Flow 取得 <br/>refresh_token]
      S2 --> Pro2[" 使用者習慣（類似 FB 連結）<br/> 可細粒度控權範圍 <br/> refresh_token 定期失效 <br/>
    end
```

Fig 8-2: 模式 A 簡單但使用者門檻高；模式 B 介面熟悉但有 token 維護成本。

百原平台**雙軌並行**：介面預設引導 OAuth（B），但在客戶 OAuth 無法取得權限時（如 Workspace 帳號有管理限制）改引導 Manager 加入（A）作為 fallback。

8.4 欄位對應表

GBP 的資料結構與 Schema.org 不完全對齊，需要一張**明確的 mapping 表**。以下列出 12 組常用對應：

GBP 欄位	Schema.org property	轉換規則
`title`	`Organization.name`	直接映射，去除前後空白
`storefrontAddress`	`Organization.address`	組成 `PostalAddress` 物件（多行地址拆解）
`primaryPhone`	`Organization.telephone`	E.164 格式正規化
`websiteUri`	`Organization.url`	驗證可解析 URL
`regularHours.periods`	`openingHoursSpecification`	週天 + 開關時間的陣列轉換
`categories.primaryCategory`	`@type` 選擇	對應到百原 25 類 industry_code
`profile.description`	`Organization.description`	最長 750 字元（GBP 限制）
`metadata.placeId`	`Organization.identifier` + `sameAs`	`identifier` 標 place_id，`sameAs` 附 Maps URL
`moreHours`	`specialOpeningHoursSpecification`	特殊時段（午休、節日）拆分
`attributes`	`amenityFeature` / `hasOfferCatalog`	依屬性類型分配不同 property
`media.photos`	`image` / `logo`	第一張設為 logo、其餘 image 陣列
`reviews`	`aggregateRating` + `review`	評分數 + 評論樣本（不全抓，取前 N 筆）

每條對應規則在 `gbpToSchema.js` 中以純函數實作；測試時以固定 fixture 驗證輸出等於預期 JSON-LD。

8.5 同步頻率與配額

GBP API 預設配額為 **300 QPM**（queries per minute）。需在「資料新鮮度」與「配額不爆」之間取捨：

Fig 8-3：同步頻率矩陣

欄位類型	同步頻率	每日 QPS	說明
基本資訊（name、address、hours）	每日 1 次	極低	變動頻率低
營業時段異動	每小時 1 次	低	臨時休假、特殊節日需及時反映
相片、屬性	每日 1 次	低	視覺內容更新頻率中等
評論	每 10 分鐘	中	評論新進較頻繁，對 aggregateRating 影響大
Q&A	每小時 1 次	低	使用者問答頻率低於評論

Fig 8-3: 每個 Location 一天約消耗 150–200 次配額；300 QPM 帳戶可支援約 2,000 Location 並行。

當品牌數量超過單帳號 QPM 上限時，策略是：

1. **分帳號** —申請 Additional QPM Quota（Google 可審核提高）
2. **分批次** —把不急的同步拆到非高峰時段
3. **分優先** —低完整度品牌優先，高完整度品牌降頻

8.6 無 Webhook 的補救方案

GBP API **不提供 webhook**[5]，無法在客戶改動 GBP 時即時通知百原。補救方案：

- **Notifications API + Pub/Sub**：Google 提供的「間接 webhook」，當 Location 有變動時會推送到 Google Pub/Sub topic，百原 subscribe 該 topic 後幾分鐘內可收到變動訊號
- **比對型同步**：每次讀取時比對 `metadata.updateTime`；只有時間更新過才重建 JSON-LD，減少不必要的 downstream 處理
- **客戶手動觸發**：UI 提供「立即同步」按鈕，讓客戶在剛改完 GBP 後立刻觸發一次拉取

這三層加總可將「GBP 改動 → AI 能見」的延遲壓到 **5–10 分鐘** 內，對絕大多數場景已足夠。若未來開放 AI 即時性更高的應用（例如「現在是否營業中」查詢），再考慮進一步最佳化。

8.7 Phase 1–4 Roadmap

Fig 8-4：GBP 整合四階段

```
flowchart LR
    P1["Phase 1<br/> 進行中 <br/> 本地 Schema.org<br/> 手動填寫 + GBP URL 貼"]
    P2["Phase 2<br/> 等 API 核准 <br/>GBP API 讀取 <br/> 自動同步基本資訊"]
    P3["Phase 3<br/> 規劃中 <br/>LocalPosts 寫入 <br/>ClaimReview 主動發布"]
    P4["Phase 4<br/> 未來 <br/>Organization Account<br/> 連鎖品牌批次管理"]
    P1 --> P2 --> P3 --> P4
```

Fig 8-4: 四階段以「讀 → 寫 → 批次」漸進鋪陳。Phase 1 不依賴 API 核准即可上線；Phase 2 在 API 核准後啟用；Phase 3–4 根據商業需求決定時程。

各階段里程碑

[5] Google Business Profile API. Notifications Overview. https://developers.google.com/my-business/content/notification-setup

階段	功能	依賴	狀態
Phase 1	25 產業 Schema.org、手動填寫、GBP URL 抽 Place ID、Wizard+Edit、AXP 注入	無外部依賴	□ 已上線（v2.19.x）
Phase 2	GBP API 讀取：基本資訊、營業時段、評論、相片自動同步	GBP API 核准	□ 審核中
Phase 3	LocalPosts 寫入（品牌公告 / 活動）、ClaimReview 主動發布到 Google	Phase 2 穩定運行 3 個月	□ 規劃
Phase 4	Organization Account：連鎖品牌一個入口管多間分店、批次編輯、統計聚合	Google 啟用 Organization Account for Partner	□ 未來

Phase 1 的完成**不需要**等 GBP API 核准—客戶貼 Maps URL 後平台抽 Place ID 存入 DB，Schema.org 即可產生 `sameAs` 連到 Google Maps。這讓平台能先上線服務而不被 Google 審核時程卡住。

本章要點

- GBP 是實體商家的事實主控源，百原平台採單向同步（GBP → Schema → AXP → AI）
- GBP API 申請需 7–10 工作天，申請理由需聚焦於「商家主管理工具」
- 代管模式雙軌：OAuth 為主、Manager 加入為 fallback
- 12 組欄位對應需明確定義，所有 mapping 以純函數實作確保可測試
- 同步頻率依欄位變動速率分級；300 QPM 配額可支援約 2,000 Location
- GBP 無 webhook，以 Notifications API + Pub/Sub 補救，延遲 5–10 分鐘
- 四階段 roadmap：Phase 1 上線、Phase 2 等核准、Phase 3–4 視需求推進

參考資料

- Ch 6 —AXP 影子文檔
- Ch 7 —Schema.org Phase 1
- Ch 9 —Closed-Loop 幻覺修復（ClaimReview 發布路徑）

Chapter 9 —Closed-Loop 幻覺偵測與自動修復

偵測幻覺只是第一步；若沒有「修復 → 驗證 → 收斂」的閉環，幻覺會像草一樣春風吹又生。

目錄

- 9.1 為什麼「偵測」不足、需要「閉環」
- 9.2 AI 幻覺的五種類型
- 9.3 偵測主機制：NLI 分類 + Chainpoll 投票
- 9.4 中央共用 RAG：SaaS 架構的關鍵基礎設施
- 9.5 L1 LLM Wiki：被動檢索之上的主動語意層
- 9.6 修復：ClaimReview 生成與多路徑注入
- 9.7 兩層掃描閉環
- 9.8 收斂時序與驗收
- 本章要點
- 參考資料

9.1 為什麼「偵測」不足、需要「閉環」

傳統品牌監測工具的邏輯是：**發現問題 → 通知客戶 → 客戶自己想辦法。**這在傳統 SEO 時代可以接受，因為問題通常是「可見性不足」——客戶自己寫新內容、發外鏈就能改善。

但 AI 幻覺不同：

- 客戶**不知道**如何修正 AI 對自己的錯誤認知
- 即使寫了「正確版本」內容，AI 也不一定會重新抓取、重新訓練
- 每個 AI 平台的資料管道不同，一處修正不代表全部修正

結論：**光把問題丟給客戶是不負責任的。**平台必須提供「從偵測到收斂」的完整自動化閉環。

Fig 9-1：閉環六階段

```
flowchart LR
    A[ 偵測 <br/> 抽取聲明] --> B[ 比對 <br/> 對照 Ground Truth]
    B --> C[ 判定 <br/> 是否為幻覺]
    C -->| 是 | D[ 生成 <br/>ClaimReview]
    D --> E[ 注入 <br/>AXP + RAG]
    E --> F[ 重掃驗證 <br/> 哨兵 + 完整]
    F -->| 收斂 | Done[結案]
    F -.->| 仍存在 | D
    C -->| 否 | Skip[不處理]
```

Fig 9-1: 從偵測到結案的六階段。任一階段失敗不影響其他阶段，系統具備局部容錯。

9.2 AI 幻覺的五種類型

百原平台將 AI 關於品牌的錯誤分成五類，各有不同的偵測與修復策略：

Fig 9-2：幻覺分類樹

```
flowchart TD
    Hallu[AI 幻覺]
```

```
    Hallu --> E1[ 實體錯置 <br/>Entity Misidentification]
    Hallu --> E2[ 業務錯誤 <br/>Business Fact Error]
    Hallu --> E3[ 產業誤分 <br/>Industry Miscategory]
    Hallu --> E4[ 時空錯誤 <br/>Temporal/Spatial Error]
    Hallu --> E5[ 屬性錯誤 <br/>Attribute Error]
    E1 -.->| 範例 | EX1[" 把台灣品牌說成日本公司"]
    E2 -.->| 範例 | EX2[" 把產品線描述成競品的"]
    E3 -.->| 範例 | EX3[" 把牙醫診所歸類成醫美"]
    E4 -.->| 範例 | EX4[" 把 2025 開業說成 2010 老店"]
    E5 -.->| 範例 | EX5[" 把 B2B SaaS 說成消費者 App"]
```

Fig 9-2: 五類幻覺並非互斥，一次 AI 回應可能同時包含多類；修復優先級依影響程度排序。

修復策略差異

類型	優先級	主要修復手段
實體錯置	P0	Schema.org `sameAs` 強化 + ClaimReview + Wikidata 連結
業務錯誤	P0	在 AXP 明示正確產品線 + ClaimReview
產業誤分	P1	修正 `industry_code` + Schema.org `@type` + RAG 同步
時空錯誤	P1	`foundingDate` / `address` 明確化 + ClaimReview
屬性錯誤	P2	強化描述、加入 FAQ、修正 `audience`

P0 類型直接影響品牌被誤認，必須最快修復；P2 多為語意偏差，較不急。

9.3 偵測主機制：NLI 分類 + Chainpoll 投票

實務上若只做「抽取 claim → 與 Ground Truth 比對」的單點查找，會遇到兩個核心問題：

1. **GT 覆蓋率低** —沒人有辦法把品牌所有事實都手動窮舉成 key-value，缺漏就產生誤報
2. **精確比對過嚴** —同一事實有多種正確表達（「成立於 2018」vs「2018 年創立」vs「已有 7 年歷史」），字串比對會把正確的說法誤判為幻覺

百原平台的偵測主機制是 **NLI（Natural Language Inference）三分類**，GT 比對只是 NLI 所需知識來源的其中一層 fallback。

三層知識來源（組合輸入 NLI）

```
flowchart TB
    Resp[AI 回應原文] --> Ext[原子事實抽取 <br/>LLM]
    Ext --> NLI[NLI 判定器]
    subgraph KS[" 知識來源（三層依權威度組合）"]
      K1["<b>L1 品牌官網 </b><br/> 即時抓取 / DB 快取 24h<br/> 第一方資料，最權威"]
      K2["<b>L2 RAG 知識庫 </b><br/> 客戶上傳 + 官方文件 <br/> 語意檢索答覆"]
      K3["<b>L3 Ground Truth</b><br/> 手動 key-value<br/> 精準但覆蓋率低"]
    end
    K1 --> NLI
    K2 --> NLI
    K3 --> NLI
    NLI --> Cls[分類結果 <br/>entailment / contradiction /<br/>neutral / opinion]
```

Fig 9-3: 知識來源權威度由高到低排列，全部組合成一段 context 餵給 NLI。若三層合計 < 500 字則跳過偵測（避免在資料稀疏的狀態誤判）。

NLI 四分類定義

類別	意義	處理
`entailment`	知識來源明確**支持** claim	事實正確、通過
`contradiction`	知識來源與 claim 明確**矛盾**	**判定為幻覺**，進入修復流程
`neutral`	知識來源**無法確認也無法否認**	不判定（重要：neutral 不是幻覺）
`opinion`	主觀評價（「最好的」「值得推薦」）	跳過，主觀不做事實查核

NLI 模型同時為每個 claim 打一個 `confidence`（0.0–1.0）與 `severity`（critical / major / minor / info）。

為何「neutral ≠ 幻覺」是核心原則

最常見的誤設計是「知識來源沒提到就視為錯誤」。但品牌官網不可能列舉所有事實，AI 說「該公司有 50 名員工」若官網沒有員工數頁面，**不代表這個數字是錯的，只代表我們無法驗證。**把 neutral 誤判為 contradiction 會大量製造假幻覺，引發錯誤的修復動作，使 AI 下一輪閱讀到被「修正」為錯誤的內容。

Chainpoll：不確定地帶的二次確認

對 `confidence` `[0.5, 0.8]` 的不確定 claim，系統啟動 **Chainpoll 多數決**：

```
// 同一 claim 以同一 prompt 呼叫 LLM 三次，取多數結果
async function chainpollVerify(claim, knowledgeContext) {
  const votes = { contradiction: 0, entailment: 0, neutral: 0 };
  const prompt = buildNLIPrompt(claim, knowledgeContext);

  const results = await Promise.allSettled([
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
  ]);

  for (const r of results) {
    if (r.status !== 'fulfilled') continue;
    const text = (r.value.text || '').toLowerCase();
    if (text.includes('contradiction')) votes.contradiction++;
    else if (text.includes('entailment')) votes.entailment++;
    else votes.neutral++;
  }

  return pickMajority(votes); // 2/3 以上才採信
}
```

Chainpoll 有效降低單次 LLM 幻覺分類本身的雜訊（畢竟分類器也是 LLM，也會有隨機性）。高 confidence（> 0.8）與低 confidence（< 0.5）的 claim 不觸發 Chainpoll，只有在**模糊地帶**才啟動，成本可控。

嚴重度分類

一旦被判定為 `contradiction`，嚴重度欄位決定修復優先級：

Severity	判定標準	修復排程
`critical`	公司名稱、產品類別、國家 / 地點完全錯誤	立即啟動修復，24h 內注入
`major`	核心功能、價格錯誤	24h 內修復
`minor`	次要功能、規格偏差	週期掃描時修復
`info`	措辭不精確	累積到一定數量才處理

這個分級讓資源集中處理**會造成商業損失**的幻覺，避免被小瑕疵淹沒。

9.4 中央共用 RAG：SaaS 架構的關鍵基礎設施

上節的 **L2 RAG 知識庫** 並非每個租戶各自建置，而是整個百原 SaaS 平台**共用一座中央 RAG 引擎**（以下以「中央 RAG」稱之）。這是很容易被工程團隊跳過但**影響深遠**的架構決策。

為何用中央共用 RAG 而非每租戶獨立 RAG

面向	每租戶獨立 RAG	中央共用 RAG（百原採用）
運維成本	每個租戶需獨立部署 / 擴容	單一集群維護
模型推論成本	每租戶單獨跑 embedding / LLM	共享 GPU/API 資源，攤提成本
知識圖譜協同	租戶間無法互相驗證	中央可做跨租戶事實交叉比對（去識別化）
升級速度	每租戶需個別升級	一處升級、全平台同步
隔離風險	隔離天然明確	需設計 tenant isolation（X-Tenant-ID + 資料權限）

百原選擇中央共用，用工程設計換取規模效率。租戶隔離透過三層機制維持：

1. **HTTP header** `X-Tenant-ID` —每次查詢帶租戶識別，RAG 引擎據此過濾文件範圍
2. **API Key 簽章** —每個 SaaS 後端呼叫 RAG 都帶 `X-RAG-API-Key`，RAG 端驗證來源
3. **RLS / 文件 ACL** —每個文件 ingest 時標記擁有者租戶，查詢時強制過濾

這三層與本平台自身的 RLS 雙保險（§2.5）類似：任何單一層疏漏都需要另外兩層同時失守才會發生跨租戶洩漏。

> **品牌層級隔離（2026-04-21 更新）**：租戶內若有多個品牌，各品牌更進一步使用獨立的 Knowledge Base ID（`rag_kb_id`）區隔知識範圍。AXP 生成時指定該品牌的 `kbId`，確保不同品牌（如同一租戶下的「醫美品牌」與「美食品牌」）的事實互不污染。新品牌啟用 AXP 時，`seedBrandRAGKB` 函數自動建立 KB 並上傳品牌 Profile 與官網頁面 URL，詳見 §6.9。

Fig 9-4：中央共用 RAG 的架構

```
flowchart LR
    subgraph Platform[" 百原 GEO SaaS（應用層）"]
      T1[Tenant A<br/> 後端]
      T2[Tenant B<br/> 後端]
      T3[Tenant N<br/> 後端]
    end
    subgraph RAG[" 中央 RAG 引擎"]
      direction TB
      Gate["API Gateway<br/>X-Tenant-ID / X-RAG-API-Key 驗證"]
```

```
    subgraph Core[" 引擎核心"]
      W["<b>L1 LLM Wiki</b><br/> 語意處理層"]
      V["<b>L2 RAG</b><br/>Vector Retrieval"]
      W --> V
    end
    Gate --> Core
  end
  T1 -->|POST /api/v1/ask<br/>X-Tenant-ID: A| Gate
  T2 -->|POST /api/v1/ask<br/>X-Tenant-ID: B| Gate
  T3 -->|POST /api/v1/ask<br/>X-Tenant-ID: N| Gate
  V --> Ans[按租戶範圍過濾的答覆]
```

Fig 9-4: 所有租戶共用同一座 RAG，但所有查詢都經 Gateway 依 Tenant ID 過濾文件範圍。

9.5 L1 LLM Wiki：被動檢索之上的主動語意層

Wiki 在本架構中不是一般意義的「文件儲存」，而是一層**由 LLM 主動處理與維護的語意知識層**。它是整個幻覺偵測能運作的**基礎設施**，值得獨立章節深入描述。

9.5.1 為何不直接對原始文件做向量檢索

一個直覺的設計是：客戶上傳的官網、FAQ、產品頁直接做 embedding、存向量 DB，查詢時直接檢索。這種純「被動 RAG」的設計有三個嚴重缺陷：

- **文件彼此矛盾無法消除** —產品頁寫「100 萬筆以上客戶」、FAQ 寫「服務超過 50 萬品牌」，向量檢索會把兩個都回傳，讓下游 NLI 看到兩個衝突的「來源」
- **時序資訊遺失** —舊版本與新版本的內容同時存在向量 DB，沒有機制判定「哪個才是現行事實」
- **摘要能力缺席** —向量檢索只會回傳「最相似的段落」，但品牌關於某個主題的完整事實可能散落在十多個段落中，檢索不會自動整合

這三個缺陷在通用 RAG 應用（如客服問答）中可容忍，在**事實查核場景**中是致命的。

9.5.2 LLM Wiki 的處理流程

LLM Wiki 是百原自建的語意處理層，對每筆 ingest 的文件執行以下處理：

Fig 9-5：LLM Wiki 的文件生命週期

```
flowchart TB
    In[新文件 ingest<br/> 客戶上傳 / 官網爬 / API 推送] --> Norm[正規化 <br/> 去格式、抽純文字]
    Norm --> Segment[語意分段 <br/> 依主題切 chunk]
    Segment --> Ext[實體抽取 <br/> 主語 / 屬性 / 事實三元組]
    Ext --> Compare{與既有 Wiki 比對}
    Compare -->| 新增事實 | Add[加入 Wiki]
    Compare -->| 矛盾 | Conflict[標記衝突 <br/> 保留最新 + 註記舊版]
    Compare -->| 補充 | Merge[合併更完整的描述]
    Add --> Reindex[增量編譯 <br/> 更新向量索引 + 知識圖譜節點]
    Conflict --> Reindex
    Merge --> Reindex
    Reindex --> Ready[對 L2 RAG 檢索可用]
```

Fig 9-5: Wiki 不只儲存文件，還主動維護「當前應被視為真實」的事實集合。任何新文件進來都會觸發對既有知識的一次再評估。

9.5.3 增量編譯（Incremental Compilation）

傳統 RAG 系統在新增 / 更新文件時，常需要「重建整個向量索引」，成本高到無法頻繁做。LLM Wiki 採用**增量編譯**：

- **變更偵測粒度**：以「事實三元組」為最小單位（例：`< 百原科技，成立於，2024>`），而非整份文件
- **只重算受影響區域**：新事實進入時，只對相關主題的向量與知識圖譜節點重算
- **版本化**：每個事實帶 `valid_from` / `valid_to` 時間戳，允許查詢特定時點的歷史狀態
- **回溯能力**：若某次 ingest 引入錯誤，可指令性 revert 至先前版本

對幻覺偵測的意義：**修復 ClaimReview 注入 Wiki 後幾分鐘內**，NLI 查詢就能使用到修正後的事實。不需等待夜間重建或週末 downtime。

9.5.4 LLM Wiki 的核心能力

LLM Wiki 以下列原生能力支援事實查核需求：

- **上下文理解**：LLM 讀取整份文件而非僅看單一段落，能回答跨段落、跨文件的綜合問題
- **引用追溯**：回答時附帶每個事實的來源段落 ID，便於下游驗證
- **多語意聚合**：同一事實的不同語意表達（「成立於 2024」vs「2024 年創立」）被視為同一節點
- **自動摘要**：長文件自動生成結構化摘要，減少後續檢索負擔
- **矛盾管理**：新舊文件衝突時保留最新版本並標記歷史，而非雙存混淆

百原透過一層薄的 REST API 把 LLM Wiki 能力暴露給業務層，對業務層隱藏底層實作細節。任何底層 LLM 或向量引擎的替換都可在此層局部進行，不影響業務層。

9.5.5 L1 Wiki 對 L2 RAG 的啟用角色

L2 RAG 的向量檢索表面上看像「標準 RAG」，但其**索引內容**來自 L1 Wiki 處理後的結構化事實，而非原始文件。這個差異意味著：

- RAG 檢索不會返回「未經 Wiki 層處理的原始段落」
- 檢索結果自帶「事實 ID」，可追溯到具體的 Wiki 節點
- 對 NLI 判定器而言，知識來源的品質遠高於直接讀取文件

換言之：**L1 Wiki 是「知識本體」，L2 RAG 是「知識檢索」。**沒有 L1，L2 只是一個更快的原始文件搜尋工具。

```
function isHallucination(claim, groundTruth) {
  const gtValue = groundTruth[claim.predicate];
  if (!gtValue) return 'unknown'; // 沒 GT 資料，不判定
  return normalize(claim.object) !== normalize(gtValue)
    ? 'confirmed'
    : 'correct';
}
```

`unknown` 狀態代表**平台沒有足夠資訊判定。**這是重要設計：**寧可標示「待驗證」，也不隨便判定為幻覺。**誤判會觸發不必要的修復動作，反而可能引入新錯誤。

9.6 修復：ClaimReview 生成與多路徑注入

ClaimReview Schema.org 範例

```
{
  "@context": "https://schema.org",
  "@type": "ClaimReview",
```

```
  "datePublished": "2026-04-18",
  "url": "https://baiyuan.io/claims/founding-year",
  "claimReviewed": " 百原科技成立於 2018 年",
  "itemReviewed": {
    "@type": "Claim",
    "appearance": "AI-generated response",
    "firstAppearance": "2026-04-16"
  },
  "reviewRating": {
    "@type": "Rating",
    "ratingValue": "1",
    "bestRating": "5",
    "alternateName": "False"
  },
  "author": {
    "@type": "Organization",
    "name": " 百原科技",
    "url": "https://baiyuan.io"
  }
}
```

ClaimReview 是 Schema.org 正式 property，Google、Facebook、Twitter 等平台都支援解析。它不是百原自定格式，而是與全球 fact-checking 生態系相容的標準[6]。

三條注入路徑

```
flowchart LR
    CR[ClaimReview] --> P1[路徑 A<br/>AXP 影子文檔]
    CR --> P2[路徑 B<br/>RAG 知識庫]
    CR --> P3[路徑 C<br/>GBP LocalPosts *Phase 3]
    P1 --> AI1[被 AI Bot 爬取]
    P2 --> AI2[被 RAG 檢索]
    P3 --> AI3[被 Google AI Overview 引用]
    style P3 stroke-dasharray: 5 5
```

Fig 9-6: 同一份 ClaimReview 同時進三條管道，最大化被 AI 重新看到的機率。路徑 C 等 GBP Phase 3 開放。

9.7 兩層掃描閉環

修復注入後，如何驗證「AI 真的改了認知」？需要重掃—但**頻率不能太低**（修復後太久才驗證，使用者失去信任），也**不能太高**（成本爆）。百原設計**兩層掃描分工**：

Fig 9-7：兩層掃描分工矩陣

```
flowchart TB
    subgraph L1["Layer 1: 哨兵掃描 (4h 週期)"]
      T1[針對搜尋型 AI<br/>Perplexity / ChatGPT Search /<br/>Google AI Overview]
      T2[輕量：抽查 5 個指標 query]
      T3[目的：快速驗證 <br/> 修復是否被抓取]
    end
```

[6] Schema.org. ClaimReview. https://schema.org/ClaimReview

```
    subgraph L2["Layer 2: 完整掃描 (24h 週期)"]
      F1[針對知識型 AI<br/>Claude / Gemini / DeepSeek /<br/>Kimi / GPT-4o 等]
      F2[深度: 20–60 個 intent query<br/>+ 七維度評分 + 指紋比對]
      F3[目的: 確認語料整合、<br/> 收斂判定]
    end
```

Fig 9-7: Layer 1 週期短、針對「會即時抓網」的 AI；Layer 2 週期長、針對「訓練週期長」的 AI。兩層共同構成完整驗收。

為何搜尋型與知識型分層

- **搜尋型 AI**（Perplexity、ChatGPT Search、AI Overview）會在回答當下實時抓取網路內容。修復注入到 AXP 幾分鐘內就能被抓到；**哨兵 4 小時驗證足夠**。
- **知識型 AI**（標準版 Claude / Gemini / DeepSeek 等）依賴預訓練語料或定期重訓的 RAG 檢索。修復注入後需要**以天計**才能被語料整合；**24 小時完整掃描**是合理頻率。

強行用統一頻率會造成：搜尋型驗收延遲（本來 4h 就能知道，等到 24h 太慢）、知識型誤報（4h 內根本沒來得及抓，誤判為修復失敗）。分層是兩類平台根本不同的必然設計。

Layer 1 與 Layer 2 資料共享

兩層的結果都寫入同一張 `scan_results` 表，透過 `scan_layer` 欄位區分。下游分析（儀表板趨勢、競品比較）會依需求**合併或分離**兩層資料：

- **合併**場景：使用者看「今日引用率」時，兩層都是同一時段的訊號
- **分離**場景：分析「修復收斂速度」時，Layer 1 與 Layer 2 收斂時程不同，需分別計算

9.8 收斂時序與驗收

Fig 9-8：一次典型修復的收斂曲線（示意）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Hallucination Prevalence after Remediation (illustrative)"
    x-axis ["Day 0", "Day 1", "Day 3", "Day 7", "Day 14", "Day 21", "Day 30"]
    y-axis "Hallucination Rate (%)" 0 --> 100
    bar [80, 75, 60, 40, 22, 10, 3]
```

Fig 9-8: 搜尋型 AI 通常 1–3 天收斂；知識型 AI 通常 2–4 週收斂。圖為聚合示意，個案時程差異大。

收斂判定規則

一個幻覺標記為「已收斂」需滿足：

1. **連續 N 次掃描**（N 由嚴謹度決定，通常 3–5）該 claim 在所有目標平台均未再出現
2. **等效 claim 變體**（同義改寫、翻譯、縮寫形式）也都未出現
3. **Layer 1 + Layer 2 都驗證過一輪**（不能只有單層通過）

滿足後 UI 顯示「□ 已修復」，客戶端記錄並附上收斂時間。

未收斂的處理

若超過預期收斂時程仍存在，系統升級為「**頑固幻覺**」：

- 分析未命中的平台特性（該平台是否依賴特定語料來源）
- 檢查 ClaimReview 是否有技術錯誤（URL 失效、格式錯誤）

- 考慮追加修復手段（Wikidata edit、Wikipedia 事實標註、LinkedIn 公司描述更新）

頑固幻覺的處理往往需要人工介入，這是目前自動化閉環的邊界。誠實承認這個邊界，比假裝全自動更負責任。

本章要點

- Closed-Loop 把「偵測、比對、判定、生成、注入、驗證」六階段全部自動化
- AI 幻覺分五類（實體錯置 / 業務錯誤 / 產業誤分 / 時空錯誤 / 屬性錯誤），各有修復優先級
- **主機制是 NLI 三分類 + Chainpoll 投票**，GT 比對僅為三層知識來源之一的 fallback 層
- 「neutral ≠ 幻覺」是核心原則：無足夠資訊驗證時寧可不判定，避免誤觸發修復
- 中央共用 RAG SaaS 架構：所有租戶共用單一 RAG 引擎，以 Tenant ID / API Key / 文件 ACL 三層機制做隔離
- **L1 LLM Wiki** 是主動語意層：處理矛盾、增量編譯、版本化、自動摘要
- L1 Wiki 是「知識本體」、L2 RAG 是「知識檢索」；沒有 L1，L2 只是更快的文件搜尋
- ClaimReview 注入 AXP + RAG + GBP LocalPosts 三條路徑；Wiki 增量編譯讓修復幾分鐘內生效
- 兩層掃描分工（Layer 1 哨兵 4h / Layer 2 完整 24h）對應搜尋型與知識型 AI 特性
- 頑固幻覺仍需人工介入，此為目前自動化的邊界

參考資料

- Ch 6 —AXP 影子文檔
- Ch 7 —Schema.org Phase 1
- Ch 10 —Phase 基線測試

Chapter 10 —Phase 基線測試：以固定問題集做縱向 AI 認知比較

每次都生不同的問題來問，你永遠不知道分數變化是來自「品牌變了」還是「問題變了」。

目錄

- 10.1 常規掃描的縱向追蹤盲點
- 10.2 Phase 基線測試設計
- 10.3 與常規掃描的資料路徑區隔
- 10.4 四類觀察軸
- 10.5 實務操作指引
- 本章要點
- 參考資料

10.1 常規掃描的縱向追蹤盲點

Ch 2 介紹的常規掃描每次都**動態生成 intent query**，目的是模擬真實使用者會問的各種問題。這個設計適合「橫向」回答「這個品牌現在被提及多少」，但**無法回答縱向問題**：「品牌上週到這週，AI 對它的認知有什麼變化?」

原因：兩次掃描的 query 集合不同，分數差異至少有三種可能：

1. **真的變化** —AI 對品牌的認知改變（可能是好是壞）
2. **query 變了** —新 query 比舊 query 容易 / 不容易觸發提及
3. **隨機性** —同一 query 在同一 AI 兩次回應不一定完全相同

若無法區隔三種來源，趨勢圖就只是**雜訊**。要過濾出真實變化，需要**固定 query 集合 + 多次重測**。

Fig 10-1：常規掃描 vs 基線測試

```
flowchart TB
    subgraph Regular[" 常規掃描（橫向）"]
      R1[每次生新 query]
      R2[答「現在幾分」]
      R3[不適合時序比較]
    end
    subgraph Baseline["Phase 基線測試（縱向）"]
      B1[固定 query set]
      B2[相隔週/雙週重測]
      B3[答「認知如何演變」]
    end
    Regular --- Baseline
```

Fig 10-1: 兩種掃描回答不同類型的問題，互補而非替代。

10.2 Phase 基線測試設計

Phase 的定義與節奏

- **Phase 1**（Day 0）：建立基線
  - 系統依品牌產業產生 **20 題** 代表性 intent query

  - 所有問題與完整 AI 回應 response_text 存入 `baseline_test_runs.queries_json` 與 `baseline_test_responses` 表
  - 這 20 題**永久保存**，不受任何後續清理機制影響
- **Phase 2**（Day 7 ±1）：第一次重測
  - 取 Phase 1 的同一組 20 題，重新問一次所有平台
  - 每題回應存成對比資料
  - 計算「相同問題、不同時間」的分數變化
- **Phase 3**（Day 14 ±1）：第二次重測
  - 同樣 20 題、第三次提問
  - 三點資料點就能畫出短期趨勢線
- **後續**（可選）：依客戶需求延伸至 Phase 4、5（月度追蹤）

Fig 10-2：三階段資料結構

```
flowchart LR
    Q[" 意圖 Query × 20（固定）"] --> P1[Phase 1<br/>Day 0]
    Q --> P2[Phase 2<br/>Day 7]
    Q --> P3[Phase 3<br/>Day 14]
    P1 -->| 對比 | P2
    P2 -->| 對比 | P3
    P1 -->| 對比 | P3
    P1 --> R[response_text × platforms]
    P2 --> R
    P3 --> R
```

Fig 10-2: 三次提問、同一組 query、三份完整 response 對比。

為何是 20 題而非更多

- **統計角度**：20 題已能涵蓋主要 intent 類別（最佳選擇 / 比較 / 推薦 / 特定場景），多出的邊際效益遞減
- **成本角度**：20 題 × 15 平台 × 3 Phase = 900 次 API 呼叫 / 品牌；再多會讓配額吃緊
- **使用者視覺化**：三條以上的 query 對比曲線已經複雜，再多難以閱讀

20 題是經驗值，未來若數據支持調整可再迭代。

10.3 與常規掃描的資料路徑區隔

Phase 基線測試走**完全獨立的資料路徑**，與常規掃描不交疊：

差異面向	常規掃描	Phase 基線
Query 來源	每次動態生成	固定於 Phase 1
觸發頻率	daily / 4h	週期性手動觸發或排程
是否計入 GEO 主分數	是	**否**（獨立呈現）
是否受 Stale Carry-Forward 影響	是	**否**（遇失敗標 incomplete）
資料保存	依滾動視窗	**永久保存 response_text**
是否走 Redis 快取	是（減少重複 API）	**否**（每次新鮮呼叫）

為何 Phase 基線不走快取

常規掃描會把同一題的近期回應快取（假設短時間內 AI 不會改意見）以降低成本。但 Phase 基線的目的就是「測量 AI 意見的變化」——若快取，就測不到變化。

為何 Phase 基線不計入主分數

若納入主分數，Phase 2 與 Phase 3 的重測會產生**三倍計入效應**（同品牌在相近時段被重複計分），污染儀表板趨勢。分離兩者保持分數純淨。

10.4 四類觀察軸

Phase 基線資料的價值不僅是「分數變化」，還有四類獨立觀察軸：

Fig 10-3：四類變化觀察矩陣

```
flowchart TB
    subgraph Quant["  量的變化"]
      Q1[引用率上升/下降]
      Q2[位置前移/後退]
      Q3[平台覆蓋擴/縮]
    end
    subgraph Qual["  質的變化"]
      L1[描述字句變化]
      L2[用詞深度變化]
      L3[新增或刪除的要點]
    end
    subgraph Comp["  競品的變化"]
      C1[共現競品名單]
      C2[競品位置關係]
      C3[新競品出現]
    end
    subgraph Sent["  情感的變化"]
      S1[情感分數位移]
      S2[強烈語氣的消長]
      S3[中性 → 正/負的轉向]
    end
```

Fig 10-3: 四類軸互不重疊；量可計算、質需質性分析、競品需圖譜對比、情感需模型打分。

各軸的分析手法

量：直接以分數差值、百分比變化、趨勢斜率計算。

質：將 Phase 1 與 Phase 2 的 response_text 做 diff 比對；突出「新增段落」「刪除段落」「替換字句」三類。高亮顯示給客戶看。

競品：抽取回應中的所有品牌實體，對比兩次 Phase 的集合差集（新進 / 退出 / 維持）。

情感：對每句話跑情感分類，比較情感分布的位移。例如 Phase 1 中性 80% / 正面 15% / 負面 5%，Phase 2 中性 60% / 正面 30% / 負面 10% 就是明顯的情感分化。

10.5 實務操作指引

何時啟用 Phase 基線

推薦三個觸發場景：

1. **客戶訂閱高階方案**時自動建立 Phase 1，使用者可在儀表板看縱向演變
2. **客戶做重大內容改版**後人工觸發 Phase 1，衡量改版效益
3. **Closed-Loop 修復**（Ch 9）後啟動 Phase 測試，驗證幻覺是否真的收斂

基線失效與重建

當下列事件發生時，Phase 基線應該**重建**而非繼續延伸：

- 品牌**業務重大變更**（併購、分拆、轉型）→ 舊 query 已不再適用
- AI 平台**發布重大版本**（如 GPT-5、Claude 4）→ 兩版本間分數不可直接比較
- Phase 基線**執行超過 6 個月**未重建 → query 與現實偏差累積

重建時建立新 `baseline_cohort_id`，舊的保留供歷史參考但不再新增資料點。

UI 呈現原則

- **不與常規儀表板混放** —避免使用者誤把 Phase 分數與日常分數相加
- **獨立頁面** `/baseline` 呈現 Phase 1→2→3 的對比視圖
- **核心視圖**：同一題三次回應並排（左中右三欄），變化處以顏色高亮
- **次要視圖**：四類觀察軸的聚合趨勢圖

本章要點

- 常規掃描動態生成 query、適合橫向「現況」；基線測試固定 query、適合縱向「演變」
- Phase 1→2→3 以 20 題重測三次，建立短期趨勢線；response_text 永久保存
- 基線測試走獨立資料路徑，不計入主分數、不走快取、不受 Stale Carry-Forward 影響
- 四類觀察軸：量（分數）、質（文字）、競品（實體集合）、情感（分布位移）
- 啟用時機：高階方案自動 / 客戶重大改版 / Closed-Loop 修復後驗證
- 基線失效重建條件：業務變更、AI 重大版本、執行超過 6 個月

參考資料

- Ch 2 —系統總覽
- Ch 3 —七維度評分演算法
- Ch 4 —Stale Carry-Forward
- Ch 9 —Closed-Loop 幻覺修復

Chapter 11 —5 品牌實戰觀察：6 週運營的匿名化數據

理論再美，數據才能驗證。以下是百原 GEO 從 2026-03 初到 4 月中、運營 5 個上線品牌約 6 週後的聚合觀察；客戶名、可辨識數字全部去識別化。

目錄

- 11.1 品牌畫像（匿名）
- 11.2 GEO 分數分布
- 11.3 平台覆蓋差異
- 11.4 Schema.org 完整度與引用率的相關性
- 11.5 AXP 部署前後對比
- 11.6 客戶端常見踩坑
- 11.7 三個意外發現
- 11.8 上線首月的商業驗證
- 本章要點
- 參考資料

11.1 品牌畫像（匿名）

5 個上線品牌涵蓋 B2B、B2C、實體、純線上四種組合：

代號	產業類型	屬性	語言市場	進場時 GEO 分
Brand A	B2B SaaS（行銷科技）	線上	中/英雙語	中段
Brand B	專業金融服務	線上	英語為主	高段
Brand C	B2B SaaS（知識管理）	線上	中/英雙語	中段
Brand D	餐飲實體連鎖	實體	中文	低段
Brand E	百原科技本身（dogfooding）	線上	中/英雙語	低段（冷啟動）

進場時 GEO 分以「低/中/高段」代替具體數字；保留相對關係、隱去絕對值。

為何選這 5 個作觀察樣本

- **Brand A / C** 作為「可比較組」：同為 B2B SaaS，中英雙語，進場條件近似，對比後續優化差異
- **Brand B** 是高段品牌起點，觀察「已經有認知的品牌還能不能再優化」
- **Brand D** 是實體商家唯一樣本，測試 GBP + Schema.org LocalBusiness 組合的效果
- **Brand E** 是冷啟動極端案例：完全新品牌、無任何外部引用

樣本數只有 5 個，**不足以做統計推論**。本章呈現的是「觀察」而非「結論」，目的是讓讀者感受真實的運作型態。

11.2 GEO 分數分布

6 週內，所有品牌七維度指標都有明顯變動。聚合後的分布型態：

Fig 11-1：5 品牌七維度雷達（週 1 vs 週 6 聚合匿名）

```
%%{init: {'theme':'base'}}%%
graph TD
    W1[" 週 1 聚合平均 <br/>Citation: 中低 <br/>Position: 中 <br/>Coverage: 低 <br/>Breadth: 低 <br/>Sent
    W6[" 週 6 聚合平均 <br/>Citation: 中 <br/>Position: 中高 <br/>Coverage: 中 <br/>Breadth: 中 <br/>Sent
    W1 -.->|6 週運營 | W6
```

Fig 11-1: 每個維度均有提升，Coverage 與 Breadth 提升幅度最大。資料以「低/中/高」三段呈現，迴避具體分數。

觀察到的三個型態

1. **Position 先動、Citation 後動**：優化 Schema.org 後，AI 提到品牌的「位置」會先前移（從末段移到前段），然後**過幾週**才反映在提及次數上升
2. **Coverage 擴得比 Breadth 快**：新增 intent query 類型的覆蓋率（如「比較題」「推薦題」）比擴充新 AI 平台容易
3. **Consistency 最晚改善**：品牌在 ChatGPT 與 DeepSeek 的認知要收斂到一致，通常需要 4–8 週

11.3 平台覆蓋差異

同一品牌在不同 AI 平台的引用率差異驚人。以 5 品牌聚合後的相對強弱：

Fig 11-2：平台覆蓋相對強度（匿名）

```
flowchart LR
    subgraph HighLang[" 英語強項品牌 (Brand A/B/C/E)"]
      H1[ChatGPT  ]
      H2[Claude  ]
      H3[Perplexity  ]
      H4[DeepSeek  ]
      H5[Kimi  ]
    end
    subgraph LocalLang[" 中文在地品牌 (Brand D)"]
      L1[ChatGPT  ]
      L2[Perplexity  ]
      L3[AI Overview  ]
      L4[DeepSeek  ]
      L5[Kimi  ]
    end
```

Fig 11-2: □□ = 引用率顯著 / □ = 有提及 / □ = 近乎零。英語 B2B 品牌在美系 AI 表現好；中文在地品牌在中國模型與 Google AI Overview 表現好。

啟發

- **不要在「不是你戰場」的平台上投入過多優化** —Brand A 在 DeepSeek 一直是零引用率，但在 ChatGPT 穩定被提及；與其硬打 DeepSeek，不如深化英語市場的 ChatGPT 優勢
- **在地品牌不應忽略中文模型** —Brand D 早期以為 ChatGPT 才是主戰場，但實際上 Kimi、DeepSeek 在台灣使用者的中文查詢表現更好，是該品牌的真正戰場
- **AI Overview 對實體商家重要性被低估** —Brand D 的使用者大量來自 Google AI Overview（而非獨立 AI app），這與 GBP 品質直接掛鉤

11.4 Schema.org 完整度與引用率的相關性

Fig 11-3：完整度 × 引用率聚合趨勢（匿名）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Completion (%) vs Citation Rate Delta after 6 Weeks (illustrative)"
    x-axis ["< 40%", "40-60%", "60-80%", "80-100%"]
    y-axis "Citation Rate Δ (relative)" 0 --> 100
    bar [10, 25, 55, 80]
```

Fig 11-3: 完整度分段對應的 6 週引用率提升幅度（以相對值呈現）。80% 以上完整度的品牌引用率提升最顯著。

觀察

- **完整度與引用率提升呈正相關**，但**不是線性**：< 60% 的品牌提升緩慢，跨過 60% 後提升加速
- 這暗示存在**「可辨識門檻」**：AI 可能需要看到一定量的結構化事實才開始在回答中主動引用該實體
- 但**完整度 > 80% 後邊際效益遞減**：把完整度從 85 → 100 不會帶來等比例的引用率提升

對運營的啟發

- **優先把所有品牌推過 60% 門檻**，比把少數品牌推到 100% 有效率
- 客戶常見的「追求 100%」焦慮，實際上是無謂的；80% 完整度加上**內容品質**比 100% 機械填空有效
- 完整度本身不是目的，**是被 AI 識別的手段**。手段達成後即可進入下一個槓桿

11.5 AXP 部署前後對比

5 品牌中，3 個是在第 2–3 週才部署 AXP（Brand A/B/C），2 個是從第 1 週就啟用（Brand D/E）。透過這個時間差可以觀察 AXP 的獨立效益：

Fig 11-4：AXP 部署前後 AI Bot 流量變化（聚合匿名）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "AI Bot Daily Visits (before vs after AXP, relative index)"
    x-axis ["Wk 1", "Wk 2", "Wk 3 (AXP deploy)", "Wk 4", "Wk 5", "Wk 6"]
    y-axis "AI Bot Visits (indexed)" 0 --> 200
    bar [25, 30, 35, 95, 140, 165]
```

Fig 11-4: 部署 AXP 的週次 AI Bot 流量開始快速上升。index 100 = 部署前平均值。

觀察

- **AXP 部署後 2 週內 AI Bot 訪問量上升 3–5 倍**（多數來自 GPTBot、ClaudeBot、PerplexityBot）
- **引用率提升滯後流量提升約 2–3 週**：AI Bot 抓到內容後，要等下一輪語料整合才能反映到回答中
- **GSC 索引量同步上升**：這是 SEO 側的意外紅利，AXP 的乾淨 HTML 對傳統搜尋引擎也友好

但：AI Bot 流量上升不等於引用率必然上升。若 AXP 內容本身語意稀薄，AI 抓到也無從參考。AXP 是**讓品牌有機會被看見**的基礎設施，不是銀彈。

11.6 客戶端常見踩坑

5 個品牌的客戶端操作，累積出五類最常見的踩坑：

Fig 11-5：踩坑類型分布（聚合匿名）

```
pie title "Customer-Side Pitfalls (5 brands, 6 weeks)"
    " 未填 industry_code" : 28
    "logo_url 缺或失效" : 22
    " 沒連 GBP (實體類) " : 18
    "description 過短 (< 20 字) " : 17
    "sameAs 外部連結未填" : 15
```

Fig 11-5: 沒選產業分類是最普遍的踩坑，影響 Schema.org `@type` 無法精準。

背後的共同問題

- **客戶不知道哪些欄位對 AI 最重要** —UI 若不明示優先級，多數客戶會按畫面順序填寫，忽略權重高的欄位
- **logo_url 的失效率出乎意料**：很多客戶貼的是內網路徑、暫時性 CDN 或已搬家的圖床，AXP 產出時會 404
- **實體商家不習慣手動連 GBP**：Brand D 是唯一的實體樣本，它用了 3 週才完成 GBP 連結；半數時間耗在 Place ID 取得過程的誤解

對 UI 設計的反饋

基於上述觀察，我們在產品端做了以下調整：

- 完整度 Banner **突顯「尚缺的高權重欄位」**，而非所有缺欄位平均呈現
- logo_url 欄位加入**即時驗證**（貼入後立刻 HEAD 請求確認 200）
- 實體商家的 GBP 連結步驟，增加**帶有截圖的分步指引**

11.7 三個意外發現

6 週運營中，有三個最初沒預期到、但值得記錄的觀察：

1. 中英雙語品牌在「跨語言一致性」上意外脆弱

Brand A / C / E 都是中英雙語，但 AI 對同一品牌在中文與英文查詢中的描述**常常差很多**。例：

- 英文查詢說「某公司是 B2B 行銷自動化平台」
- 中文查詢說「某公司是行銷顧問服務」

這不是幻覺（兩個描述都部分對），而是 **AI 對跨語言同一實體的聚合能力仍不成熟**。對品牌主的啟發：中英文版 Schema.org 要**顯式用 `sameAs` 互指**，並確保兩版描述語意等價，而非各寫各的。

2. 「無引用」比「負面引用」更難處理

預期中「AI 說品牌壞話」是最大威脅，實際運營發現**「AI 根本不提」**是更大的問題。負面提及至少代表 AI 知道這個實體存在、可以用 ClaimReview 修正；完全沒被提及則代表**品牌不在 AI 的候選池**，沒有修復的著力點。

這解釋了為何 Ch 3 的 Citation Rate 權重設在 25% —既要反映它的重要性，又不讓它主宰總分使 Sentiment 等其他維度被邊緣化。

3. 競品共現可以意外地成為優勢訊號

原本以為「同一回答同時出現競品」是負面（分散注意力），但觀察發現：**跟對的競品同列，反而增強品牌的 category 認知**。例如 Brand A 在早期與兩家知名大廠共現，雖然 Citation Rate 不算高，但在使用者的心智中被放到「同一等級」的位置，後續轉換率意外好。

這個發現還需要更多數據驗證，但它說明 GEO 的「敵友」關係與傳統 SEO 的「競爭者」直覺可能是相反的——**在 AI 時代，被與對的品牌放在一起，比被單獨提及更有價值。**

11.8 上線首月的商業驗證

上述 5 個品牌是**內部 dogfooding + 合作夥伴 pilot** 的混合樣本。與此同時，百原 GEO 作為付費 SaaS 在**正式上線的第一個月內**已簽下三組商業客戶，涵蓋與 pilot 不同的產業型態：

產業	型態	選擇百原 GEO 的主要驅動
連鎖醫美	多分店實體 + 高競爭產業	GBP 整合 + 實體商家 Schema.org 特化 + 幻覺偵測（醫療類容錯低）
新興連鎖餐飲	多分店實體 + 快速擴張期	Location 級別獨立 AXP + Phase 基線測試捕捉擴張期變化
頂級芳療 / 瑜珈	高客單價、口碑型	Content Depth 與 Sentiment 雙維度優先；引用敘事品質甚於頻次

觀察

- **三組客戶的選擇標準各不相同**，印證了七維度評分（Ch 3）的設計假設：不同產業關心的維度不同，單一指標無法服務全市場
- **連鎖醫美與連鎖餐飲的共同需求是「多分店」**—這是 Phase 4 Organization Account（§8.7）提前被商業推動的訊號
- **頂級芳療 / 瑜珈的需求讓 Content Depth 維度從「次要」升級為「首要」**—高客單價產業的決策鏈長、敘事品質比頻次關鍵

這三組客戶在本書撰寫時（2026-04-18）才剛上線，具體運營數據會在未來修訂版中補上。列在這裡的意義是：**百原 GEO 的工程設計不只是內部驗證可行，對外付費市場也驗證了其需求真實性。**

本章要點

- 5 個品牌 6 週觀察屬「觀察」非「結論」；樣本數不足以做統計推論
- 七維度全面提升，但 Position 先動、Citation 後動；Consistency 最晚收斂
- 平台覆蓋有語言與產業偏食性，應集中於「真正的戰場」而非全平台硬打
- Schema.org 完整度與引用率正相關但非線性，60% 是可辨識門檻，80% 後邊際遞減
- AXP 部署後 AI Bot 流量 3–5 倍，引用率提升滯後約 2–3 週；流量 ≠ 引用率
- 客戶端五類最常踩坑（產業碼 / logo / GBP / description / sameAs），UI 以「高權重突顯」反饋
- 三個意外發現：跨語言一致性脆弱、「無引用」比「負評」更難處理、對的競品共現反成優勢
- 上線首月已簽連鎖醫美、新興連鎖餐飲、頂級芳療 / 瑜珈三組商業客戶，各自驅動不同維度的需求

參考資料

- Ch 3 —七維度 GEO 評分演算法
- Ch 6 —AXP 影子文檔
- Ch 7 —Schema.org Phase 1

- Ch 9 —Closed-Loop 幻覺修復

Chapter 12 —限制、未解問題與未來工作

一個工具誠實列出做不到的事，比聲稱無所不能更值得信任。

目錄

- 12.1 目前做不到的事
- 12.2 AI 模型版本變動的不可預測性
- 12.3 未解問題
- 12.4 未來工作 Roadmap
- 12.5 給同業與研究者的邀請
- 本章要點
- 參考資料

12.1 目前做不到的事

Fig 12-1：現況覆蓋矩陣

功能維度	覆蓋程度	缺口
監測	完整	僅限支援的 15 個 AI 平台；其他如 Claude Projects、私有部署的 LLM 無法觸及
評分	完整	跨產業比大小無意義；查詢空間主觀
結構化資料	完整	多語版 Schema.org 僅 zh-TW + en；日、韓、東南亞語系待擴充
幻覺偵測	部分	依賴知識來源品質；知識稀疏時覆蓋率下降
幻覺修復	部分	頑固幻覺需人工介入
自動化閉環	部分	搜尋型收斂快、知識型收斂慢，中間態難以完整回饋
外部平台驗證	受限	LinkedIn、Crunchbase、G2、Capterra 無公開 API，僅能手動
GBP 整合	受限	Phase 2 API 核准中；目前僅能靠 URL 抽 Place ID

Fig 12-1: 「完整」= 功能齊全；「部分」= 覆蓋核心但有缺口；「受限」= 受外部因素限制。

具體限制清單

- **無法 100% 驗證所有外部平台**：LinkedIn、Crunchbase 等無公開 API 的平台，只能手動確認存在與否，無法程式化比對內容正確性
- **GEO 分數不是絕對值**：不同產業的查詢空間不同，跨產業比大小並無意義。只在同產業、同時間段、相同查詢池下的比較才有效

- **中文 AI 模型覆蓋率落後英文**：中文 LLM 對非主流品牌的認知深度仍明顯低於英文模型；百原透過強化 L1 LLM Wiki 的中文事實結構化部分緩解，但無法根本解決
- **無 Webhook 機制**：多數 AI 平台不提供「被提及即通知」的 event；依賴輪詢會有延遲

12.2 AI 模型版本變動的不可預測性

這是一個**無法從工程端完全解決**的問題：當 OpenAI 發布 GPT-5、Anthropic 發布 Claude 4、或 DeepSeek 發布新主力模型時，所有品牌的分數可能同時跳動 3–10 分。

三類版本變動對分數的影響

類型	範例	影響方向
主要模型升級	GPT-4o → GPT-5	多數品牌分數上升（新模型訓練資料更新）
安全 / 對齊調整	某家模型拒答率變嚴	多數品牌分數下降（被 refusal 蓋掉引用）
檢索增強啟用 / 關閉	Claude 新增 / 關閉 web search	不同品牌方向相反，取決於該品牌內容的網路可得性

緩解策略

百原無法阻止這些變動，但透過以下三個機制降低對使用者的衝擊：

1. **版本敏感期告知**：偵測到 AI 平台主要版本切換時，UI 顯示「資料正在適應新模型，短期波動屬常態」的 banner
2. **Phase 基線測試的跨版本標記**：新舊模型版本間的基線資料**不可直接比大小**，UI 明確區隔
3. **等權重歷史對比**：內部保留「特定版本下的分數」供趨勢分析，不把版本躍變視為品牌變化

12.3 未解問題

1. 真實負評 vs 幻覺錯誤

當 AI 說「該品牌客服很差」時，這可能是：

- **幻覺**：AI 從不相關來源誤聯想
- **真評**：有真實使用者負評流到訓練語料

兩種情況的處理截然不同：幻覺應修復；真評應改善服務而非掩蓋。目前百原的自動化**無法可靠區分兩者**，需要人工介入判斷來源。這是影響閉環完整性的重要缺口。

2. 因果 vs 相關

客戶做了內容改版，3 週後引用率上升。這是：

- **因果**：內容改版直接提升 AI 認知
- **相關**：同期有其他外部事件（新聞、業配、季節性）帶動

目前無法嚴謹證明因果。學術上要解決這個問題需要 A/B 測試基礎（同一品牌分半、一半改版一半不改），但商業上不可能。這是 **GEO 工具共同的研究性缺口。**

3. 長尾查詢的覆蓋策略

意圖查詢動態生成的 20–60 題能覆蓋主要 intent，但**長尾查詢**（使用者問的非常具體、不常見的問題）無法窮舉。當客戶反映「我的使用者問了 XX，AI 沒提我」，這到底是：

- 隨機抽樣下本來就抓不到的長尾
- 還是系統性的覆蓋漏洞

目前只能個案處理。未來若引入「客戶自定 intent query」功能可緩解，但會面臨「客戶只敢問對自己有利的問題」的偏差風險。

12.4 未來工作 Roadmap

Fig 12-2：未來功能依賴圖

```
flowchart LR
    subgraph Short[" 短期 (6 個月內) "]
      A1[GBP API Phase 2-3<br/> 資料讀取與寫入]
      A2[多語 Schema.org<br/> 擴充到 ja / ko]
      A3[視覺化趨勢圖 <br/> 加強 Phase 基線呈現]
    end
    subgraph Mid[" 中期 (6-12 個月) "]
      B1[更多 AI 平台 <br/>Mistral / Cohere 深化 <br/>+ Claude Projects]
      B2[跨語言 sameAs<br/> 自動化填寫]
      B3[競品共現智慧建議]
    end
    subgraph Long[" 長期 (12 個月 +) "]
      C1[因果推論研究 <br/>A/B 測試方法論]
      C2[私有部署 LLM<br/> 的實體監測]
      C3[多租戶客製 intent query]
    end
    A1 --> B1
    A2 --> B2
    A3 --> C3
```

Fig 12-2: 三階段 roadmap。每階段依前階段交付；具體時程依需求與 Google/各 AI 平台的外部進度調整。

短期重點

- **GBP API Phase 2–3** 是目前等核准中的最大單一項；Phase 2（讀取）預計 1–2 個月內啟用、Phase 3（LocalPosts 寫入）需在 Phase 2 穩定運行 3 個月後才啟用
- **多語 Schema.org** 針對日、韓市場的客戶需求；語言模型可直接複用既有 API，只需擴充 i18n 內容與 @type 對應
- **Phase 基線視覺化加強** 是客戶反饋中最頻繁的 UX 項

長期目標

- **因果推論研究**：與學術機構合作，發表 GEO 領域的 A/B 測試方法論，成為該領域的共同基礎
- **私有部署 LLM 監測**：服務於企業客戶內部 AI 應用（如客服 bot、內部知識助手）的實體認知監測，是 SaaS 之外的潛在產品線

12.5 給同業與研究者的邀請

本書試圖做到的是**讓 GEO 成為一個可被討論、可被共同推進的工程學科**，而非單一工具商的閉源經驗。為此：

- **歡迎引用、改寫、翻譯本書的任何章節**（依 CC BY-NC 4.0 授權）
- **歡迎在 GitHub Issues 提出勘誤、質疑、補充**（見 README 的 Issue Templates）
- **歡迎提出共同研究合作**：特別是因果推論、跨語言實體對齊、頑固幻覺等學術性議題
- **歡迎複用架構模式**：Stale Carry-Forward、多 Provider 容錯、中央共用 RAG、閉環修復等通用工程設計

GEO 還在非常早期。希望本書是這個領域**早期的開放技術文件**之一，讓後來的團隊能從我們已經踩過的坑上起步，而不是每個人都重蹈覆轍。

本章要點

- 百原 GEO 有明確限制：外部平台驗證受限、中文模型覆蓋落後、無 Webhook 機制
- AI 模型版本變動不可預測，透過告知、基線切片、等權對比三機制緩解
- 三個未解問題：真評 vs 幻覺、因果 vs 相關、長尾查詢覆蓋
- Roadmap 分短（GBP API 完成）、中（跨語 sameAs）、長（因果推論）三階段
- 邀請同業共同推進：引用本書、GitHub 討論、研究合作、架構複用

參考資料

- Ch 3 —七維度評分演算法（分數限制的深入討論）
- Ch 8 —GBP API Phase Roadmap
- Ch 9 —Closed-Loop 修復邊界
- Ch 11 —實戰觀察與意外發現
- 倉庫 README —貢獻與引用方式

Appendix A —詞彙表（完整版）

依字母 / 漢字筆劃排序。每條列出：**定義 + 本書首次出現章節 + 相關章節。**

A

- **@id（Schema.org）**—JSON-LD 中實體的唯一識別字串，用於在 `@graph` 內或跨文件互相引用。本書首見：§6.7。核心應用：§7.3。
- **AI Bot** —泛指抓取網路內容以訓練或檢索的機器程式，如 GPTBot、ClaudeBot、PerplexityBot。本書首見：§6.3。
- **AI Citation Rate（AI 引用率）**—在代表性意圖查詢中，品牌被 AI 主動提及的比例（0–100%）。本書核心指標。首見：§1.2。詳解：§3.2.1。
- **aggregateRating** —Schema.org property，聚合評分值。本平台從 GBP 同步。§8.4。
- **AXP (AI-ready eXchange Page)** —百原造詞。給 AI Bot 讀的影子文檔，pure HTML + Schema.org JSON-LD + Markdown 三層結構，與人類版網站解耦。詳述於 Ch 6。
- **App-level filter** —應用層查詢條件。本書指多租戶隔離的第二層防線（與 RLS 搭配）。§2.5。

B

- **BullMQ** —Redis 為底層的 Node.js 任務隊列函式庫。本平台的背景任務執行骨架。§2.4。
- **Brand Entity（品牌實體）**—以 Schema.org Organization / LocalBusiness 為根節點的結構化品牌資料總稱。Ch 7。

C

- **ChainPoll** —對同一 claim 以同一 prompt 呼叫 LLM 3 次取多數決的驗證方法；用於 NLI confidence 0.5–0.8 不確定地帶。§9.3。
- **CID（Customer ID）**—Google Maps 內部流水號，需透過 Places API 反查成 Place ID。§7.7。
- **ClaimReview** —Schema.org 正式 property，用於標注「某聲明是否為真」。本平台用於對外宣告幻覺修復結果。§9.6。
- **Closed-Loop（閉環修復）**—偵測 → 比對 → 判定 → 生成 ClaimReview → 注入 AXP/RAG → 重掃驗證的自動化循環。Ch 9。
- **Cloudflare Workers** —邊緣計算環境，本平台用於 AI Bot UA 偵測與 AXP 注入。§6.3。
- **Common Crawl** —公開的大規模網頁抓取資料集，是多數 LLM 預訓練的主要資料來源之一。§1.3。
- **Consistency（跨平台一致性）**—七維度之一，同一品牌在不同 AI 平台各維度分數的標準差之倒數。§3.2.7。
- **Content Depth（內容深度）**—七維度之一，AI 提及品牌時所附帶描述的深度。§3.2.6。
- **Core Web Vitals** —Google 的網頁體驗指標（LCP、FID、CLS）。SEO 時代重要，GEO 時代幾無關聯。§1.3。

D

- **DIRECT_MODEL_ID_MAP** —將聚合中轉的 model ID 對應到廠商原生 model ID 的對照表。§5.3.1。
- **Dogfooding** —自家產品給自家用。本平台的 Brand E 即為範例。§11.1。

E

- **entailment / contradiction / neutral / opinion** —NLI（自然語言推論）四分類。contradiction 才標記為幻覺；neutral ≠ 幻覺。§9.3。
- **extraParams** —各 AI 廠商原生 API 特有的必填參數。例：Qwen3 需 `enable_thinking: false`。§5.3.2。

F

- **Fingerprint（指紋）**—對 AXP 或 Schema.org 產物的雜湊值，用於判斷內容是否被 AI 抓到並整合。Ch 9 相關。
- **FTID（Feature ID）**—Google Maps 內部地點識別碼。§7.7。

G

- **GBP（Google Business Profile）**—Google 商家檔案，實體商家的事實主控源。Ch 8。
- **GEO（Generative Engine Optimization）**—生成式引擎優化。提升品牌在生成式 AI 回答中被提及比例、位置、敘事準確度的學科。Ch 1。
- **GPTBot** —OpenAI 的網頁爬蟲，用於 ChatGPT 訓練語料收集。§6.4。
- **Ground Truth (GT)** —事實真相。本書指用於比對幻覺候選的權威資料集合（官網即時抓 / RAG / 手動 GT 三層）。§9.3。

H

- **Hallucination（幻覺）**—AI 生成的、與事實不符的內容。本書分五類：實體錯置 / 業務錯誤 / 產業誤分 / 時空錯誤 / 屬性錯誤。§9.2。

I

- **industry_code** —百原 25 類產業分類 enum。§7.2。
- **Intent Query（意圖查詢）**—依品牌產業動態生成的代表性使用者問題。§2.2.1。
- **is_physical** —品牌的實體 / 線上屬性旗標，影響 Schema.org 欄位權重。§7.4。
- **isStale** —Stale Carry-Forward 下的標記欄位，表示該平台的分數為歷史帶入值。§4.3。

J

- **JSON-LD** —JavaScript Object Notation for Linked Data，Schema.org 的主流序列化格式。§6.2、Ch 7。

K

- **kgmid（Knowledge Graph Machine ID）**—Google 知識圖譜的實體識別。在 `sameAs` 中使用。
- **Knowledge Graph（知識圖譜）**—實體與關係構成的結構化網絡。Schema.org 的 `@id` 互連建立此圖。§7.3。

L

- **Layer 1 哨兵掃描** —4 小時週期、針對搜尋型 AI 的輕量驗證。§9.7。
- **Layer 2 完整掃描** —24 小時週期、含七維度評分與指紋比對的深度掃描。§9.7。
- **LLM Wiki** —百原造詞。由 LLM 主動處理與維護的語意知識層，位於 RAG 系統的 L1 層。§9.5。
- **LocalBusiness** —Schema.org @type，實體商家的主要分類。§7.2。

M

- **modelRouter** —百原自研的多 AI Provider 路由服務，主 / 備雙路徑抽象層。Ch 5。

N

- **NLI（Natural Language Inference）**—自然語言推論，判斷兩段文字的蘊含關係。本平台幻覺偵測主機制。§9.3。

O

- **OpenAI-compatible API** —以 OpenAI API 規格為介面的聚合中轉。多 AI 廠商可用同一 SDK 呼叫。§5.2。

P

- **pgvector** —PostgreSQL 向量擴充，支援 embedding 與相似度查詢。§2.4。
- **Phase 基線測試** —以固定 20 題在不同時間點重測同一 AI，縱向追蹤認知演變。Ch 10。
- **Place ID** —Google Maps 的地點唯一識別。Places API 的 primary key。§7.7。
- **Position Quality（位置品質）**—七維度之一，品牌在 AI 回答中出現位置的加權平均。§3.2.2。
- **Platform Breadth（平台覆蓋度）**—七維度之一，品牌被主動提及的 AI 平台數 ÷ 監測平台總數。§3.2.4。

Q

- **Query Coverage（查詢覆蓋率）**—七維度之一，被提及的 intent query 類型多樣性。§3.2.3。
- **QPM（Queries Per Minute）**—每分鐘查詢次數，API 配額計量單位。GBP API 預設 300 QPM。§8.5。

R

- **RAG（Retrieval-Augmented Generation）**—檢索增強生成。本平台採中央共用 RAG SaaS 架構。§9.4。
- **RLS（Row-Level Security）**—PostgreSQL 資料列層級存取控制。多租戶隔離的第一層。§2.5。

S

- **sameAs** —Schema.org property，指向其他平台的同一實體（Wikipedia、Wikidata、LinkedIn、GBP）。§7.3。
- **Schema.org** —結構化資料詞彙表，Google/Bing/Yahoo/Yandex 聯合推動。Ch 7。
- **Sentiment Score（情感分數）**—七維度之一，提及文本的情感傾向（正 / 中性 / 負 聚合）。§3.2.5。
- **Sentinel Scan（哨兵掃描）**—見 Layer 1。
- **SERP（Search Engine Results Page）**—搜尋引擎結果頁，傳統 SEO 的主戰場。§1.1。
- **Sitemap** —XML 格式的 URL 清單，協助爬蟲發現內容。§6.6。
- **Smoke Test** —備援路徑啟動時的最小驗證，確保可用性。§5.6.1。
- **Stale Carry-Forward** —百原造詞。當平台掃描全失敗時，從歷史帶回最近成功值並標記為「停滯」，維持訊號連續性。Ch 4。

T

- **Tenant ID（X-Tenant-ID）**—多租戶 HTTP header，用於中央共用 RAG 的租戶過濾。§9.4。

U

- **UA（User-Agent）**—HTTP 請求標頭，AXP 注入據此判斷是否為 AI Bot。§6.4。

W

- **Webhook** —伺服器主動推送變動通知。GBP API 目前**不提供** webhook。§8.6。
- **Wikidata** —開放式知識圖譜，Schema.org `sameAs` 的重要目標。§7.3。
- **Wizard（設定引導）**—新品牌的線性七步驟 Schema.org 填寫流程。§7.6。

相關規範索引

規範	作用	本書章節
Schema.org	結構化資料詞彙	Ch 6, 7, 9
JSON-LD 1.1	Linked Data 序列化	Ch 6, 7
OpenGraph	Meta tag for preview	Ch 6
robots.txt	Robots Exclusion Protocol	Ch 6
sitemaps.xml	Sitemap Protocol 0.9	§6.6

Appendix B —公開 API 規格節錄

本附錄節錄百原 GEO Platform **對外公開**的 REST endpoint 規格，供第三方整合、研究與審計參考。內部管理性質 endpoint（admin、監控、billing）不收錄。

共通規範

Base URL

`https://api.geo.baiyuan.io/v1`

Authentication

所有 endpoint 需帶 `Authorization: Bearer <token>` 標頭。取得 token 請見客戶面板「API 金鑰管理」。

Response Envelope

所有回應為：

```
{
  "status": "success" | "error",
  "data":   { ... },           // status=success 時
  "error":  "description"      // status=error 時
}
```

Rate Limit

- 預設 60 req/min；企業方案可提升
- 超限回 HTTP 429，`Retry-After` 標頭告知等待秒數

---

1. Brands

GET `/brands`

列出當前帳戶可見的品牌。

Query params：

- `limit`（1–100，預設 20）
- `offset`（預設 0）
- `industry_code`（可選，見 §7.2 的 25 類 enum）

Response：

```
{
  "status": "success",
  "data": {
    "brands": [
      {
        "id": "brd_01H...",
        "name": "Example Brand",
        "industry_code": "saas_application",
        "url": "https://example.com",
        "completion": 78,
        "current_geo_score": 62
```

```
      }
    ],
    "total": 42,
    "limit": 20,
    "offset": 0
  }
}
```

GET `/brands/:id`

取得單一品牌詳情。

POST `/brands`

建立新品牌。**body**：`name`（必填）、`url`、`industry_code`、`description` 等。

PUT `/brands/:id`

更新品牌。body 內容依 `Content-Type` 而定；支援 merge-patch 語義。

---

2. Scans

POST `/scans`

觸發一次掃描（立刻執行、非排程）。

body：

```
{
  "brand_id": "brd_01H...",
  "platforms": ["chatgpt", "claude", "perplexity"],  // optional; 預設全部
  "intent_count": 20,
  "scan_layer": "full" | "sentinel"                   // 預設 "full"
}
```

Response（202 Accepted）：

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
    "estimated_duration_seconds": 180,
    "queue_position": 3
  }
}
```

GET `/scans/:id`

查詢掃描狀態。

Response：

```
{
  "status": "success",
  "data": {
```

```
    "scan_id": "scn_01H...",
    "state": "queued" | "running" | "completed" | "failed",
    "progress": 0.75,
    "started_at": "2026-04-18T09:15:00Z",
    "completed_at": null,
    "results_url": "/scans/scn_01H.../results"
  }
}
```

GET `/scans/:id/results`

取得掃描結果。

Response（節錄）：

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
    "scan_layer": "full",
    "geo_score": 62,
    "dimensions": {
      "citation":    { "value": 55, "isStale": false },
      "position":    { "value": 70, "isStale": false },
      "coverage":    { "value": 60, "isStale": false },
      "breadth":     { "value": 58, "isStale": false },
      "sentiment":   { "value": 72, "isStale": false },
      "depth":       { "value": 50, "isStale": false },
      "consistency": { "value": 65, "isStale": false }
    },
    "platforms": {
      "chatgpt":    { "sov": 60, "position": "top-third" },
      "claude":     { "sov": 58, "position": "top-third" },
      "perplexity": { "sov": 48, "position": "middle" }
    }
  }
}
```

---

3. Entity（Schema.org 品牌實體）

GET `/brands/:id/entity`

取得品牌的 Schema.org 完整資料（含 Organization、Services、Employees、Hours、Location、Social Links）。

GET `/brands/:id/entity/schema.json`

取得生成後的 Schema.org JSON-LD（可直接嵌入客戶網站）。

Response：`Content-Type: application/ld+json`；body 為 `{"@context": "https://schema.org", "@graph": [...]}`。

GET `/brands/:id/entity/completion`

取得完整度分數。

Response：

```
{
  "status": "success",
  "data": {
    "total_percent": 78,
    "breakdown": {
      "basic_info":  { "percent": 100, "weight_pct": 25 },
      "location":    { "percent":  80, "weight_pct": 15 },
      "services":    { "percent":  60, "weight_pct": 15 },
      "employees":   { "percent":  40, "weight_pct": 10 }
    },
    "missing_high_priority": ["description", "logo_url"]
  }
}
```

---

4. AXP（影子文檔）

GET `/axp?host=<domain>&path=<path>`

此 endpoint **由 Cloudflare Worker 呼叫**，一般整合開發者無需直接使用。返回的是該品牌該路徑的影子文檔 HTML。

---

5. Hallucinations

GET `/brands/:id/hallucinations`

列出已偵測到的幻覺。

Query params：`severity`（critical/major/minor/info）、`status`（open/resolving/resolved）。

POST `/brands/:id/hallucinations/:hid/repair`

觸發修復流程。

---

6. Baseline（Phase 基線測試）

POST `/brands/:id/baseline`

建立新的 Phase 基線 cohort。

POST `/brands/:id/baseline/:cohortId/run`

對既有基線重測。

GET `/brands/:id/baseline/:cohortId/compare`

取得 Phase 1 vs Phase 2 / 3 的對比。

---

7. Webhooks（客戶可接收）

百原平台**主動推送**以下事件到客戶自設的 webhook URL（客戶於面板設定）：

事件	Trigger
`scan.completed`	一次掃描完成
`hallucination.detected`	偵測到新幻覺（severity >= major）
`hallucination.resolved`	幻覺收斂
`completion.threshold_crossed`	完整度跨過 60% / 80% / 100% 關鍵門檻
`gbp.sync_failure`	GBP 同步連續失敗 3 次

Payload 格式：

```
{
  "event": "scan.completed",
  "brand_id": "brd_01H...",
  "scan_id": "scn_01H...",
  "occurred_at": "2026-04-18T09:30:00Z",
  "data": { ... }
}
```

每個 webhook 帶有 `X-Baiyuan-Signature` 標頭（HMAC-SHA256），客戶應驗證後才處理。

錯誤代碼

HTTP	Code	說明
400	`invalid_parameter`	參數格式 / 值不合法
401	`unauthenticated`	未帶 token 或 token 失效
403	`forbidden`	帶 token 但無此資源權限
404	`not_found`	資源不存在
409	`conflict`	狀態衝突（如 scan 進行中不能重複觸發）
422	`unprocessable_entity`	資料驗證失敗
429	`rate_limited`	超過配額
500	`internal_error`	內部錯誤，請重試或聯絡支援

Appendix C —參考文獻

本書引用的所有外部資料來源。分為四類。

1. 官方規範與標準

Schema.org 與 Linked Data

- Schema.org. Schema.org Vocabulary Specification. https://schema.org/docs/schemas.html
- W3C. JSON-LD 1.1: A JSON-based Serialization for Linked Data. https://www.w3.org/TR/json-ld11/
- Schema.org. ClaimReview. https://schema.org/ClaimReview
- Schema.org. LocalBusiness. https://schema.org/LocalBusiness
- Schema.org. Organization. https://schema.org/Organization
- Schema.org. Service. https://schema.org/Service
- Schema.org. Person. https://schema.org/Person

Google 生態

- Google Business Profile API. Overview. https://developers.google.com/my-business
- Google Business Profile API. Notifications Overview. https://developers.google.com/my-business/content/notification-setup
- Google Maps Platform. Places API documentation. https://developers.google.com/maps/documentation/places/web-service
- Google Search Central. How Search works. https://www.google.com/search/howsearchworks/
- Google Search Central. Fact check article markup. https://developers.google.com/search/docs/appearance/structured-data/factcheck
- Google Search Central. Google-Extended and generative AI training. https://developers.google.com/search/docs/crawling-indexing/overview-google-crawlers

AI Bot User Agents

- OpenAI. GPTBot User Agent Documentation. https://platform.openai.com/docs/gptbot
- OpenAI. OAI-SearchBot User Agent. https://platform.openai.com/docs/bots
- Anthropic. ClaudeBot documentation. https://support.anthropic.com/en/articles/8896518
- Perplexity. PerplexityBot. https://docs.perplexity.ai/guides/bots
- Common Crawl. CCBot User Agent. https://commoncrawl.org/ccbot

網站通訊協定

- IETF. RFC 9309 —Robots Exclusion Protocol. https://www.rfc-editor.org/rfc/rfc9309.html
- sitemaps.org. Sitemap Protocol 0.9. https://www.sitemaps.org/protocol.html
- W3C. HTTP/1.1. https://www.rfc-editor.org/rfc/rfc7230

開放授權

- Creative Commons. CC BY-NC 4.0 Legal Code. https://creativecommons.org/licenses/by-nc/4.0/legalcode
- Citation File Format. CFF 1.2.0 Specification. https://github.com/citation-file-format/citation-file-format

---

2. 技術框架與開發工具

- Cloudflare. Workers Runtime Documentation. https://developers.cloudflare.com/workers/
- Next.js. App Router & Server Components. https://nextjs.org/docs
- PostgreSQL. Row Security Policies. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- pgvector. Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector
- BullMQ. Queue documentation. https://docs.bullmq.io/
- OpenAI. Node.js SDK. https://github.com/openai/openai-node
- Anthropic. TypeScript SDK. https://github.com/anthropics/anthropic-sdk-typescript

---

3. 產業研究與統計資料

- OpenAI. (2025). Usage & Revenue Update, Q3 2025. 官方季度揭露（引用於 Ch 1）
- Perplexity AI. (2025). Year in Review 2024: Search Volume & Engagement. 官方部落格（引用於 Ch 1）
- SimilarWeb. (2025). The State of Generative Search: AI Overview Impact on Publisher Traffic. 研究報告（引用於 Ch 1）
- Google. (2024). Generative AI in Search: Let Google do the searching for you. 官方公告

---

4. 書籍與專書

- Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O’Reilly Media. ISBN 978-1449373320.
  - 第 8 章「The Trouble with Distributed Systems」對部分失敗處理的討論是 Ch 4 Stale Carry-Forward 的通用參考框架。
- Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2nd ed.). Pragmatic Bookshelf. ISBN 978-1680502398.
  - Circuit Breaker、Bulkhead、Timeout 三個 pattern 是 Ch 5 modelRouter 的設計參考。
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution. ISBN 978-1942788003.
  - 閉環回饋設計（Ch 9）受第三之路「持續學習」章節啟發。

---

5. 相關學術研究（選錄）

本書撰寫時未做完整文獻回顧。以下為作者閱讀過、與本書主題相關的代表性研究；歡迎讀者於 GitHub Issues 補充推薦。

LLM 幻覺偵測

- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023.
- Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. EMNLP 2023.

知識圖譜與 Entity Linking

- Pan, J. Z., Razniewski, S., Kalo, J.-C., et al. (2024). Large Language Models and Knowledge Graphs: Opportunities and Challenges. TGDK 2024.
- Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). Knowledge Graphs. ACM Computing Surveys, 54(4).

多源證據的 NLI

- Nie, Y., Chen, H., & Bansal, M. (2019). Combining Fact Extraction and Verification with Neural Semantic Matching Networks. AAAI 2019.
- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: A Large-scale Dataset for Fact Extraction and VERification. NAACL 2018.

結構化資料對 LLM 的影響

- Yang, J., Chen, H., Yan, Y., et al. (2024). Exploring the Role of Structured Data in Large Language Model Fact Verification. 預印本。

如何引用本書

本書的正式引用格式請見 `CITATION.cff` 或 README 的引用方式段落。

```bibtex
@techreport{lin2026baiyuangeo,
  author      = {Lin, Vincent},
  title       = {Baiyuan GEO Platform: A Whitepaper on Building a SaaS for Generative Engine Optimizatio
  institution = {Baiyuan Technology},
  year        = {2026},
  url         = {https://github.com/baiyuan-tech/geo-whitepaper},
  note        = {v1.0}
}
```

Appendix D —配圖總表與製圖規格

本書所有圖表集中索引。Mermaid 圖於 `.md` 原始檔內嵌、GitHub 原生渲染；其他靜態圖（若有）放 `assets/figures/`。

D.1 總表

編號	類型	主題	章節
Fig 1-1	xychart-beta	2023–2025 搜尋管道流量遷徙趨勢	Ch 1
Fig 1-2	對照表	SEO vs GEO 核心差異	Ch 1 §1.3
Fig 2-1	flowchart	閉環系統六階段	Ch 2 §2.1
Fig 2-2	flowchart	品牌治理完整循環	Ch 2 §2.3
Fig 2-3	flowchart	多租戶 RLS + App-level filter 雙保險	Ch 2 §2.5
Fig 3-1	graph	七維度雷達關聯示意	Ch 3 §3.2
Fig 4-1	xychart-beta	三策略訊號對比（直接計 0 / Stale Carry-Forward）	Ch 4 §4.3
Fig 4-2	stateDiagram	平台資料狀態機 (Fresh/Stale/Expired/Reset)	Ch 4 §4.3
Fig 4-3	ASCII	前端 isStale 徽章示意	Ch 4 §4.6
Fig 5-1	flowchart	modelRouter 雙路徑架構	Ch 5 §5.2
Fig 5-2	flowchart	切換觸發決策樹	Ch 5 §5.5
Fig 6-1	flowchart	同品牌兩種視角（人類版 vs AI 版）	Ch 6 §6.1
Fig 6-2	flowchart	AXP 文件三層結構	Ch 6 §6.2
Fig 6-3	flowchart	Cloudflare Worker 路由決策流程	Ch 6 §6.3
Fig 6-4	flowchart	25 種 AI Bot UA 四類分群	Ch 6 §6.4
Fig 6-5	flowchart	SaaS 自家品牌路徑分類決策樹	Ch 6 §6.5
Fig 6-6	code block	JSON-LD flat vs nested 對比	Ch 6 §6.7
Fig 7-1	table	25 產業分類表（實體 16 + 線上 7 + 保底 2）	Ch 7 §7.2
Fig 7-2	flowchart	三層 @id 知識圖譜	Ch 7 §7.3
Fig 7-3	flowchart	實體 vs 線上欄位權重分歧	Ch 7 §7.4
Fig 7-4	flowchart	Wizard + Edit 雙入口流程	Ch 7 §7.6
Fig 7-5	flowchart	GBP URL Parser 四分支決策樹	Ch 7 §7.7
Fig 8-1	flowchart	資料單向流：GBP → Schema → AXP → AI	Ch 8 §8.1
Fig 8-2	flowchart	兩種代管模式對比 (Manager / OAuth)	Ch 8 §8.3
Fig 8-3	table	同步頻率矩陣	Ch 8 §8.5
Fig 8-4	flowchart	GBP 整合四階段 Roadmap	Ch 8 §8.7

編號	類型	主題	章節
Fig 9-1	flowchart	閉環六階段	Ch 9 §9.1
Fig 9-2	flowchart	幻覺五類分類樹	Ch 9 §9.2
Fig 9-3	flowchart	三層知識來源餵 NLI	Ch 9 §9.3
Fig 9-4	flowchart	中央共用 RAG 架構	Ch 9 §9.4
Fig 9-5	flowchart	LLM Wiki 文件生命週期	Ch 9 §9.5
Fig 9-6	flowchart	ClaimReview 三條注入路徑	Ch 9 §9.6
Fig 9-7	flowchart	兩層掃描分工矩陣	Ch 9 §9.7
Fig 9-8	xychart-beta	幻覺 prevalence 收斂曲線	Ch 9 §9.8
Fig 10-1	flowchart	常規掃描 vs 基線測試	Ch 10 §10.1
Fig 10-2	flowchart	Phase 1/2/3 資料結構	Ch 10 §10.2
Fig 10-3	flowchart	四類變化觀察矩陣	Ch 10 §10.4
Fig 11-1	graph	5 品牌七維度雷達 (週 1 vs 週 6)	Ch 11 §11.2
Fig 11-2	flowchart	平台覆蓋相對強度（英語強項 vs 中文在地）	Ch 11 §11.3
Fig 11-3	xychart-beta	完整度 × 引用率聚合趨勢	Ch 11 §11.4
Fig 11-4	xychart-beta	AXP 部署前後 AI Bot 流量	Ch 11 §11.5
Fig 11-5	pie	客戶端踩坑類型分布	Ch 11 §11.6
Fig 12-1	table	現況覆蓋矩陣	Ch 12 §12.1
Fig 12-2	flowchart	未來功能依賴圖 (短/中/長期)	Ch 12 §12.4

總計：13 章共 44 張圖（Mermaid 主導，含部分 ASCII art、table、code block）。

D.2 製圖準則

1. 風格統一

全書使用以下調色盤（Mermaid `%%{init: {'theme':'base'}}%%` 後再微調）：

用途	色碼
主色（節點邊框、重點）	`#1e40af`（深藍）
輔色（強調、箭頭）	`#ea580c`（亮橘）
警示（錯誤、失敗路徑）	`#dc2626`（紅）
成功（修復、收斂）	`#16a34a`（綠）
中性（背景、輔助）	`#64748b`（灰藍）

Mermaid 的 `style` 語法可在必要節點套用以上色碼：

```
flowchart LR
    A[節點]
    style A fill:#1e40af,color:#fff
```

2. 圖優先使用 Mermaid

- **技術流程圖**（flowchart / stateDiagram / sequenceDiagram）一律 Mermaid
- **資料關係圖**（graph、ER diagram）Mermaid
- **趨勢圖**（xychart-beta）Mermaid
- **pie chart** Mermaid

理由：

- 可版控、可 diff、可由讀者複製再製
- 不依賴外部圖床（圖床失效會造成白皮書壞圖）
- AI 爬蟲能讀 Mermaid 原始 code，圖本身也是結構化資料

3. 靜態圖（SVG/PNG）的使用時機

僅在以下情況使用靜態圖，放 `assets/figures/`：

- Mermaid 無法表達的複雜視覺（多層嵌套、藝術化圖示）
- 產品截圖（UI 畫面、客戶面板示範）
- 實際資料圖表需要 Recharts 互動性（PDF 版降為靜態 PNG）

所有靜態圖必須：

- 同時保留原始檔（`.fig`、`.drawio`、Figma 連結）
- 以 SVG 為首選格式（可縮放、檔案小）
- 檔名採 `fig-<chapter>-<number>-<slug>.svg` 格式，例：`fig-11-03-completion-citation.svg`

4. 圖片 alt text 規範

所有靜態圖必須有**描述性 alt text**，格式：

`![Fig X-Y 主題簡述：具體內容 <br>*Fig X-Y: 詳細 caption*](assets/figures/fig-XX-YY-slug.svg)`

alt text 須能讓**不看圖的讀者理解圖的主旨**（同時服務視障使用者與 AI 爬蟲）。

5. 中英雙版的圖獨立製作

- 中文版的圖**不直譯**成英文版；英文版重新繪製，符合英文語境慣例
- 數據單位、日期格式、文字方向依語系調整
- 兩版共用編號（Fig 1-1 中文版 / Fig 1-1 英文版同對應）

6. 產品截圖的去識別化

UI 截圖必須：

- 客戶品牌名打碼（用 `Brand A`、`Example Brand` 替代）
- 真實數字模糊或替換為示例（例：`$12,345` → `$XX,XXX`）
- 內部 URL 路徑打碼（僅顯示結構，不顯示具體域名）
- Email 地址遮蔽（僅顯示 `user@***.com` 形式）

7. Mermaid 圖的相容性

Mermaid 於 GitHub Markdown、GitLab、Obsidian、VS Code Preview 均原生支援。但**在 Pandoc PDF 生成**時需要：

- CI 在 build 時用 `mermaid-cli` 將 code block 預轉為 SVG
- 或使用 Pandoc filter `pandoc-mermaid` 自動處理

`build-pdf.yml` 工作流已包含此步驟（見 `.github/workflows/build-pdf.yml`）。

D.3 製作進度

狀態	圖數量	佔比
□ 已於 `.md` 內完成	44	100%
□ 需產品實際截圖	0	0%
□ 英文版另繪	0（pending 英文版）	—